

It Is Not the Signature: A Pre-Authentication Key-Parse Attempt Dominates JSON Web Token Validation Cost and Is Coupled to a Key-Confusion Defence

Jannatul Ferdous Katha^{a,c}, Tasmia Tahmid Prova^{b,c}, Md Kishor Morol^c, Tze Hui Liew^{d,*}, Dip Nandi^{c,e}

^a*Department of CSE, Bangladesh University of Professionals (BUP), Dhaka, Bangladesh*

^b*Department of CSE, Bangladesh University of Business and Technology (BUBT), Dhaka, Bangladesh*

^c*ELITE Research Lab, Queens, NY, USA*

^d*Faculty of Information Science and Technology, Multimedia University, Melaka, Malaysia*

^e*Department of Computer Science, American International University-Bangladesh (AIUB), Dhaka, Bangladesh*

Abstract

JSON Web Token (JWT) validation is the authentication step of most micro-service requests, and its per-request cost is routinely attributed to cryptography. In `jsonwebtoken`, a widely used Node.js implementation, the signature primitive is 2.208 μ s of a 29.92 μ s validation. The dominant component, 18.04 μ s (60%), is a discarded key-parse attempt: a shared secret supplied as a string is first offered to the asymmetric key parser and is resolved only after that parse fails. The attempt runs before the algorithm check and before signature verification, so any well-formed token, forged or not, triggers it. It entered the library with the security release that fixed an RSA-to-HMAC key-confusion vulnerability, and it is load-bearing: removing it the obvious way fails the library's own key-confusion tests. Its price is set, causally, by the bundled OpenSSL release: the same calls issued from C cost 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8. In a deployed service, validation adds 384.6 μ s of processor time per request, within 0.8% of the microbenchmark

*Corresponding author.

Email addresses: `ferdous.katha@elitelab.ai` (Jannatul Ferdous Katha), `tasmia.prova@elitelab.ai` (Tasmia Tahmid Prova), `kishormorol@ieee.org` (Md Kishor Morol), `thliew@mmu.edu.my` (Tze Hui Liew), `dip.nandi@aiub.edu` (Dip Nandi)

prediction. Of seven JWT libraries in four languages, only this one attempts the parse. We give a narrow fix that preserves the key-type defence and makes validation 4.15 times faster, show that a client controlling the token header can still reach the parse under it, and identify pre-parsed keys as the remedy that removes it for every input. In our sample, 94% of call sites pin no algorithm and none pre-parses the key. The defect has been reported upstream.

Keywords: JSON Web Token, authentication, key confusion, cryptographic API, OpenSSL, Node.js, performance measurement

1. Introduction

Authentication in cloud-native services is commonly described as imposing a per-request cryptographic cost (Zhu et al., 2023). The assumption is old and has been tested before, one layer down: Coarfa et al. (2006) instrumented TLS web servers and found RSA responsible for only a minority of server cost, the remainder falling to symmetric operations, hashing and protocol handling. Our result is that argument repeated at the application layer, and reaching a sharper version of the same conclusion.

The reasoning being tested is intuitive: validating a JSON Web Token (Jones et al., 2015) means verifying a signature, signatures are cryptography, and cryptography is expensive relative to parsing. The conclusion usually drawn is that the cost should be *moved* — to a sidecar proxy, a gateway, or dedicated hardware — because specialised components do cryptographic work well.

We report that the premise is wrong for the most widely used JavaScript implementation we could measure. The natural repair is wrong as well. Once the signature primitive is shown to be a small share of the cost, the remainder is readily attributed to key material being converted from a string into a key object on every call. Measuring that conversion directly puts it at 0.583 μ s: too small to be the explanation for anything.

The actual mechanism is a control-flow accident, shown in Figure 2. Given a secret that is not already a key object, the library attempts `createPublicKey()` first and falls back to `createSecretKey()` in the exception handler. For an HMAC secret the first call cannot succeed. Every validation therefore constructs and discards an OpenSSL parse failure, at 18.04 μ s against a total call cost of 29.92 μ s.

27 We are careful about what that 18.04 μ s is. It is the cost of the whole
28 discarded attempt: the work OpenSSL performs before deciding the input is
29 not an asymmetric key, plus the construction and unwinding of the exception
30 that carries that decision back to the library. Section 4.2 separates the two,
31 by reissuing the OpenSSL calls from C and by measuring an exception of
32 the same shape with no OpenSSL call. Where the attempt is expensive it is
33 almost all parse; the exception is a few microseconds. The title’s “key-parse
34 attempt” is therefore the accurate name, and “discarded exception” would
35 not be.

36 How much the attempt costs depends on which OpenSSL release performs
37 it, by more than an order of magnitude (Section 5). Runtimes carrying an
38 OpenSSL release from 3.2 onward, which include the current Node.js release
39 lines, pay tens of microseconds per validation; runtimes carrying the 3.0
40 series pay hundreds. The 3.0 series is not confined to retired Node.js releases.
41 It is the system OpenSSL of long-term-support distributions still in wide
42 use, among them Ubuntu 22.04 and Debian 12, and a Node.js build that
43 links the system library rather than its bundled copy inherits it; so does any
44 service whose image is pinned to an older base layer, which is common in
45 practice (Zerouali et al., 2019). At either price the discarded attempt is the
46 largest single component of the call, and in every case it is avoidable.

47 The cost is also a security matter, in three ways (Section 8). First, it is
48 paid before authentication: key resolution precedes both the algorithm check
49 and the signature check, so every request carrying a well-formed token pays it,
50 whether or not anyone the service trusts issued that token. Second, it did not
51 arrive through neglect. The resolution block entered the library in version 9.0.0,
52 the release that fixed an RSA-to-HMAC key-confusion vulnerability (Auth0,
53 2022), and the key-type checks that implement that fix read the type of the key
54 object the block produces. Third, it therefore cannot be removed the obvious
55 way: dispatching on the token’s declared algorithm without restricting which
56 key material may take the fast path fails the library’s own key-confusion tests.
57 A performance change to authentication code is a change to its security, and
58 we analyse it as one.

59 The result is broader than one library in two respects. The design that
60 produces the cost — resolving key material on every call, by way of a
61 parse whose failure serves as a type test — can be named independently of
62 `jsonwebtoken`, and Section 7 shows it predicts which of seven libraries pay.
63 And the reason the cost went unattributed is methodological: the component
64 that sets its price, the bundled OpenSSL release, is one that measurement

65 protocols rarely record at the granularity at which it changes.

66 *Research questions.*

67 **RQ1** Where does the per-request cost of token validation actually go?

68 **RQ2** What governs the magnitude of that cost?

69 **RQ3** Does it matter in a deployed service, and how common is the pattern
70 that avoids it?

71 **RQ4** Is the defect a property of one library, or of a design choice other
72 libraries could share?

73 **RQ5** How does the cost relate to the library’s security: who can cause it, why
74 is it there, and can it be removed without weakening the key-confusion
75 defence?

76 *Contributions.* All supported by measurements in the replication pack-
77 age (Katha et al., 2026).

- 78 1. A decomposition of the validation call, by two independent methods,
79 locating its dominant component in a discarded key-parse attempt
80 rather than in the signature primitive or in key conversion, and a
81 second decomposition of that attempt into the OpenSSL parse, the V8
82 exception and the binding between them (Section 4).
- 83 2. A security analysis of that component (Section 8). It is reachable before
84 authentication by any well-formed token; it entered the library with a
85 key-confusion fix and is load-bearing for it, so the natural optimisation
86 fails the library’s key-confusion tests; a narrow fix preserves the defence
87 and makes validation 4.15 times faster, but leaves the parse reachable
88 through an attacker-chosen algorithm header; and only a pre-parsed
89 key removes it for every input.
- 90 3. A causal attribution of its magnitude to the bundled OpenSSL release:
91 the parse reissued from C with no JavaScript engine across 8 identically
92 built OpenSSL releases, a runtime pair that crosses engine and library,
93 and a repetition on a second architecture (Section 5).
- 94 4. A deployed-service measurement across seven arrival rates, and a survey
95 establishing that the avoiding pattern is rare in three samples of public
96 code (Section 6).
- 97 5. A comparison of seven JWT libraries in four languages showing that
98 the cost follows a design choice — resolving key material on every call,
99 of which a failing asymmetric parse used as a type test is the costliest
100 form — rather than JWT validation as such (Section 7).

101 2. Background and related work

102 *What a validation call does.* A JSON Web Token carries a header, a payload
103 and a signature over the first two (Jones et al., 2015). Validating one means
104 decoding two segments, resolving key material into a form the cryptographic
105 library accepts, and verifying the signature. The middle step — key resolution
106 — is the subject of this paper, and it is the step most often left implicit when
107 validation cost is discussed.

108 *Where the cost is usually attributed.* Published measurement of layered en-
109 forcement in microservices concentrates on transport and proxy layers. Zhu
110 et al. (2023) decompose service-mesh sidecar overhead and show that the dom-
111 inant contributors depend on configuration and workload rather than being a
112 fixed tax; we reach an analogous conclusion one layer up, for application-level
113 validation. Application-level token validation is itself thinly evidenced, and is
114 commonly reported as a difference between scenario averages, which attributes
115 to validation every difference between two runs. We were unable to locate
116 a study that measures the key-resolution step directly; we state that as an
117 absence we searched for rather than as a claim about the whole literature.

118 *Cryptographic cost has been over-attributed before.* Coarfa et al. (2006) remains
119 the clearest precedent: measuring TLS web servers rather than primitives,
120 they found the public-key operation was a minority of server cost. Work on
121 cryptographic library performance since has largely stayed at the level of
122 the primitive or the protocol exchange (Naser et al., 2020; Sikeridis et al.,
123 2020), with Tuveri and Brumley (2019) a notable exception in examining how
124 a library loads and dispatches implementations rather than how fast those
125 implementations run. Our result concerns neither: it is the cost of getting
126 key material *into* the library.

127 *Performance bugs as a class.* Jin et al. (2012) studied real-world performance
128 bugs across five code bases and found that most follow a small number of
129 recurring patterns — among them uncoordinated and skippable function calls
130 — and that fixes are usually local. Zaman et al. (2012) found, from 400 Firefox
131 and Chrome reports, that performance bugs differ from other bugs in how
132 they are reported, discussed and fixed, in part because reproducing them is
133 hard; Nistor et al. (2013a) report that most are found by code reasoning rather
134 than by observing symptoms. Liu et al. (2014) reach similar conclusions for
135 mobile applications. Detection work targets structural symptoms: redundant

136 traversals with similar memory-access patterns (Nistor et al., 2013b), or
137 predicates that separate slow from fast runs (Song and Lu, 2014). The defect
138 here is of the “skippable call” kind Jin et al. describe, but it has none of the
139 structural signatures those detectors look for: it is one call, executed once
140 per request, whose failure is its only observable effect. It is also the shape
141 that makes a performance bug hard to report: our own upstream report was
142 preceded by a two-year-old report of the symptom with no diagnosed cause
143 (Section 8.6).

144 *JavaScript and Node.js performance.* Selakovic and Pradel (2016) classified
145 performance issues in JavaScript projects and found that inefficient API usage
146 — correct calls made in a costly way — is among the most common root causes,
147 and that the fixes are typically a few lines. Their later profiler (Selakovic et al.,
148 2017) targets precisely the *order* in which a program evaluates conditions, the
149 axis on which this defect lives: a check that almost always fails is evaluated
150 before one that almost always succeeds. Gong et al. (2015) locate code that
151 defeats JIT optimisation; exception-heavy control flow is one such pattern,
152 and Section 4.2 measures what it costs here. Work on the Node.js event loop
153 shows why a per-request synchronous cost matters beyond its magnitude: a
154 single slow handler delays every other client (Davis et al., 2018; Staicu and
155 Pradel, 2018). We make no availability or capacity claim, but the mechanism
156 by which a synchronous per-request cost propagates is the same.

157 *Version and environment variation.* That software performance changes
158 across versions in ways maintainers do not anticipate is established: Kalte-
159 necker et al. (2023) track performance across configurable systems’ release
160 histories, and Chen and Shang (2017) study the code changes that introduce
161 regressions. Laaber and Leitner (2018) assess whether microbenchmark suites
162 detect such changes at all, and Stefan et al. (2017) find performance unit
163 testing rare in practice. Container images make this concrete by pinning a
164 dependency set implicitly: Cito et al. (2017) characterise how the ecosystem
165 uses them, Zerouali et al. (2019) show that images routinely ship outdated
166 packages, and Malka et al. (2026) show that an image tag does not guarantee
167 a reproducible build. For OpenSSL 3 specifically, the maintainers of a major
168 downstream library have stated publicly that parsing and key loading re-
169 gressed substantially against 1.1.1 (Gaynor and Kehrer, 2026); that statement
170 is not peer reviewed and was not produced under a protocol we can assess,
171 and Section 5.5 supplies a controlled measurement for the one path this paper
172 concerns. Section 5 is an instance with a measured cost attached.

173 *Measurement protocol is the thing at stake.* The methodological core of this
174 paper is not new. Georges et al. (2007) established that a single execution
175 of a managed-runtime benchmark is not a measurement, and that intervals
176 must be computed across executions rather than within one; Kalibera and
177 Jones (2013) show how to spend a fixed budget across levels of repetition,
178 which is the reasoning behind our choice of many short processes over few
179 long ones. Mytkowicz et al. (2009) showed that environmental details unre-
180 related to the code under study can produce effects larger than the one being
181 measured, without any obvious mistake, and Curtsinger and Berger (2013)
182 that memory layout alone can do so. Barrett et al. (2017) found that virtual
183 machines frequently do not reach a steady state, Traini et al. (2023) that
184 developers misjudge when warmup ends, and Costa et al. (2021) catalogue how
185 microbenchmarks go wrong in practice. Mytkowicz et al. (2010) showed that
186 sampling profilers disagree on which method is hot, which is why Section 4
187 uses the profiler only as corroboration of a result obtained by differencing and
188 relies on ordering rather than on its percentages. Our contribution is not to
189 these methods but is an instance of what they warn about. The attribution
190 this paper overturns has exactly the structure Mytkowicz et al. describe:
191 a second variable — the bundled OpenSSL version — moving unrecorded
192 alongside the one under study, at a granularity coarse enough to hide it
193 (Section 5).

194 *Environment as a confounder.* Laaber et al. (2019) quantify how far cloud en-
195vironments degrade microbenchmark reliability, and Leitner and Cito (2016)
196 characterise performance variation across public infrastructure providers.
197 Their concern is variance between nominally identical environments. Ours is
198 adjacent and, we think, less examined: a *systematic* difference between envi-
199ronments that are not identical and are routinely treated as though they were
200 — two container base images differing in a bundled library version. Laaber
201 et al. are also the reason the second-architecture repetition in Section 5.6 is
202 reported as an ordering check rather than as a replication of magnitudes.

203 *Cryptographic interfaces are hard to use correctly.* The defect reported here
204 is not a cryptographic weakness but a consequence of how a cryptographic
205 interface is invoked, which is a well-documented class. Lazar et al. (2014)
206 found that most of the cryptographic vulnerabilities they studied were misuses
207 of libraries by applications rather than bugs in the libraries. Egele et al. (2013)
208 found cryptographic misuse widespread in deployed applications and Georgiev

et al. (2012) found certificate validation broken across a wide range of non-browser software; Nadi et al. (2016) attribute much of it to the interfaces themselves rather than to developer inattention, a position Green and Smith (2016) put directly and Indela et al. (2016) respond to with interface proposals; and Acar et al. (2017) show experimentally that API design materially changes whether developers produce correct code. Detection has been automated by specifying correct usage, as CrySL does (Krüger et al., 2018), and by static analysis at scale (Rahaman et al., 2019). Our finding sits within that line with one difference: the interface here admits a use that is *correct*, and costly only for a reason invisible at the call site. A CrySL-style rule could express “pass a key object, not a string”, but the rule sets we know of encode security properties only, so none would report it.

JWT and JOSE libraries. Security analyses of JWT implementations consistently find key-handling defects. Detering et al. (2017) demonstrated attacks, key confusion among them, against several popular JOSE libraries; Xu et al. (2023) automated the detection of cryptographic vulnerabilities in applications that use JWT; Yang et al. (2026) evaluated 43 JWT libraries across 16 programming languages for vulnerabilities; and Shatnawi et al. (2024) surveyed Python JWT libraries for security warnings and adoption. RFC 8725 (Sheffer et al., 2020) codifies the resulting practice. All of this work treats a library’s handling of key material as a security surface. Section 7 treats the same surface as a cost site, across seven libraries, and Section 8.4 shows the two views are coupled: the control flow that costs time in one library is the control flow that enforces its key-type check.

Mining public repositories. Section 6.5 draws on code hosted publicly, and the hazards of doing so are catalogued by Kalliamvakou et al. (2014) — among them that many repositories are not software projects, that activity is not uniformly distributed, and that a search result is not a random sample. Our sampling frame is stated in Section 3.9 with those hazards in view, which is why no population proportion is computed from it.

The cost of exceptions. That exceptions are expensive in managed runtimes, and that their cost is concentrated in construction rather than in unwinding, is long established: Ogasawara et al. (2006) optimise around exception paths in a Java virtual machine, and Renwick et al. (2019) quantify the cost in a compiled setting. Parravicini and Mueller (2021) provide the closest available cost-attribution baseline for the V8 engine, though on speculation rather than

245 exceptions. Empirical studies of how exceptions are *used* find handlers that
 246 discard what they catch to be common: Cabral and Marques (2007) report
 247 that handlers in the Java and .NET code they studied rarely attempt recovery,
 248 Kery et al. (2016) find that handlers mostly log, print, return or rethrow, and
 249 de Pádua and Shang (2017) measure the prevalence of handling anti-patterns,
 250 as Weimer and Necula (2004) did earlier for error-handling mistakes. That
 251 literature treats a swallowed exception as a correctness and maintainability
 252 smell. The handler at the centre of this paper is a swallowed exception used
 253 as control flow on a hot path, and it is a performance defect rather than
 254 a correctness one: the fallback it guards is correct. None of these studies
 255 measures such a handler’s cost; Section 4.2 does, and finds the exception itself
 256 the smaller part.

257 *Key-type confusion.* That a token declaring an HMAC algorithm must not be
 258 validated against asymmetric key material is long established; RFC 8725 (Shef-
 259 fer et al., 2020) describes the attack and its mitigations. The library studied
 260 here has been hardened against this class twice. Versions before 4.2.2 accepted
 261 an HMAC token verified with an RSA or EC public key used as the shared
 262 secret (CVE-2015-9235) (GitHub Advisory Database, 2015), and versions
 263 up to 8.5.1 could still be led to verify forged tokens from RSA to HMAC
 264 through their key-retrieval path (CVE-2022-23541), alongside two related
 265 defects in key-type and default-algorithm handling (CVE-2022-23539, CVE-
 266 2022-23540) (Auth0, 2022); all three were fixed in version 9.0.0. Section 8
 267 reports no new attack. It reports that the second hardening introduced the
 268 control flow whose cost this paper measures, that the defence depends on
 269 that control flow, and that removing the cost naively removes the defence.

270 2.1. What we searched for and did not find

271 Table 1 summarises the positioning. Each cell records only what this
 272 section states about the work cited; it is a map of the lines of work this paper
 273 connects, not an assessment of them.

274 Three areas bear directly on this result and we were unable to locate
 275 verifiable peer-reviewed work in any of them. We state each as an absence
 276 we searched for rather than as a claim about the whole literature; each is
 277 also, in part, why the misattribution corrected here survived as long as it did.
 278 Two adjacent questions — the cost of error construction in V8, and where
 279 key-parsing cost lies inside OpenSSL — were equally unstudied, and this
 280 paper measures both directly (Sections 4.2 and 5.5).

Table 1: Positioning against the closest lines of work. *JWT*: studies JWT or JOSE implementations. *Crypto-API security*: treats how a cryptographic interface or key material is used as a security concern. *Cost*: studies a performance cost or performance defect. *Version as cause*: attributes a cost or defect to the version of a bundled dependency. *Cost-defence coupling*: shows that removing a cost would weaken a security defence. ✓: addressed; ◦: partly (see note); -: not addressed.

Work	JWT	Crypto-API security	Cost	Version as cause	Cost-defence coupling
Detering et al. (2017); Xu et al. (2023); Yang et al. (2026)	✓	✓	—	—	—
Egele et al. (2013); Nadi et al. (2016); Krüger et al. (2018); Rahaman et al. (2019)	—	✓	—	—	—
Coarfa et al. (2006); Naser et al. (2020); Sikeridis et al. (2020)	—	—	✓	—	—
Zhu et al. (2023)	—	—	✓	—	—
Jin et al. (2012); Selakovic and Pradel (2016)	—	—	✓	—	—
Kaltenecker et al. (2023); Chen and Shang (2017)	—	—	✓	◦ ^a	—
Zerouali et al. (2019); Cito et al. (2017)	—	—	—	◦ ^b	—
This paper	✓	✓	✓	✓	✓

^aVersions of the system under study, not of a bundled dependency. ^bContainer images pin outdated dependency versions; no cost is measured.

281 *OpenSSL 3.x performance regressions.* We found no peer-reviewed study
282 of performance regressions across the OpenSSL 3.x series. Evidence exists
283 as a maintainer statement (Gaynor and Kehrer, 2026), as project issues —
284 for instance `openssl/openssl` issues #17627 and #20698, which concern
285 performance of the 3.x provider and decoder machinery — and as mailing-list
286 discussion. None was produced under a measurement protocol we can assess,
287 so Section 5.5 measures the path directly rather than adopting the mechanism
288 those sources propose.

289 *Base image to bundled library to application performance.* We found
290 no academic study tracing that chain. Malka et al. (2026) address build
291 reproducibility, Zerouali et al. (2019) package staleness and Cito et al. (2017)
292 ecosystem practice; none measures how the library version an image pins

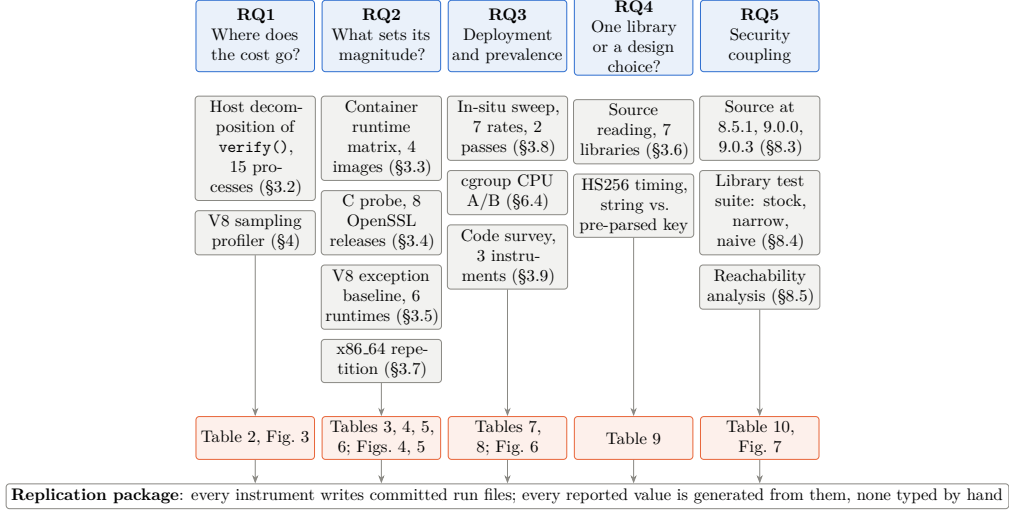


Figure 1: Study workflow. Top: the five research questions. Middle: the instruments that answer each, with the section describing it. Bottom: the tables and figures each produces. The C probe and the exception baseline (RQ2) also decompose the discarded attempt for RQ1 (Table 3), and the call-site survey (RQ3) supplies the algorithm-pinning figure used in RQ5. All measurements feed one replication package, from which every reported value is generated.

293 changes application performance.

294 *Cryptographic API misuse as a performance defect.* The misuse literature
295 cited above treats misuse exclusively as a security problem. We found no
296 work treating a correct-but-costly use of a cryptographic interface as a latency
297 or throughput defect, which is precisely what this paper reports.

298 3. Methodology

299 Figure 1 gives the study’s workflow: the instruments used for each research
300 question and the tables and figures each produces. Every instrument writes
301 to committed run files, from which every number in the paper is generated
302 (Data availability).

303 3.1. Unit of measurement

304 The unit is one operating-system process, not one call. Each process
305 performs 40000 timed iterations of a single condition after a warmup of half
306 that number, and contributes its median. Reported figures are medians of
307 per-process medians, with percentile bootstrap intervals taken over processes.

308 An interval computed over the calls inside one process describes that
309 process’s steady state and is far too narrow: it cannot observe the compilation
310 decisions, inline-cache states and heap layouts that differ between runs of the
311 same code (Georges et al., 2007). Conditions in the decomposition of Section 4
312 were run in 15 separate processes of 40000 calls; conditions in the runtime
313 matrix of Section 5 in 10 processes of 20000 calls. The largest between-process
314 coefficient of variation is 3.5% on the host; in the matrix it is 20.2%, occurring
315 on the `probe_succeeds` condition under `node:26.6.0-bookworm` rather than
316 across the matrix generally.

317 Bootstrap intervals over 15 or 10 units are coarse, and we report them
318 as indicative rather than as calibrated coverage. Where an interval collapses
319 to a point — the timer overhead row of Table 2 — it should be read as the
320 clock’s resolution showing through, not as a precise estimate. The per-process
321 values underlying every interval are in the replication package.

322 Conditions are interleaved within each round rather than run in blocks.
323 Under block ordering a thermal excursion or a background process lands
324 entirely on whichever condition is executing and is indistinguishable from an
325 effect; interleaved, it is distributed across conditions and appears as between-
326 round variance. The clock’s own cost is measured in the same process as the
327 condition it accompanies and is reported rather than subtracted: 0.042 μ s
328 against the smallest quantity reported, 0.583 μ s.

329 Differences between conditions are reported throughout as the median
330 of per-round *paired* differences, not as the difference of two medians. The
331 two are not identical and the distinction matters at the magnitudes involved;
332 Section 4.3 and Section 8.4 each rely on it.

333 3.2. Decomposing the call

334 `jwt.verify()` is decomposed into the operations it performs, each timed
335 in isolation under the same protocol: structural decode; the HMAC primitive
336 with a string key and with a pre-parsed key; the symmetric key constructor;
337 and the asymmetric key parse the library attempts first, in both the case
338 where it throws and the case where it succeeds.

339 The library measured throughout is `jsonwebtoken` version 9.0.3, cloned
340 at tag v9.0.3. The code path at issue is shown in Figure 2.

341 3.3. Separating the runtime from the deployment

342 A host microbenchmark compared against an in-container measurement
343 cannot support an attribution to deployment: those two environments differ

```

120 if (secretOrPublicKey != null && !(secretOrPublicKey instanceof
    KeyObject)) {
121   try {
122     secretOrPublicKey = createPublicKey(secretOrPublicKey);
123   } catch (_) {
124     try {
125       secretOrPublicKey = createSecretKey(typeof
        secretOrPublicKey === 'string' ? Buffer.from(
        secretOrPublicKey) : secretOrPublicKey);
126     } catch (_) {
127       return done(new JsonWebTokenError('secretOrPublicKey is not
        valid key material'))
128     }
129   }
130 }

```

Figure 2: Key resolution in jsonwebtoken version 9.0.3, verify.js, lines 120--130. Verbatim but for the common leading indentation, which is removed to fit the column; line numbers are the file's own. A secret that is not already a `KeyObject` is offered to `createPublicKey()` first, and `createSecretKey()` is reached only from the exception handler. For an HMAC secret supplied as a string the outer `try` can never succeed, so the asymmetric parse is attempted and discarded on every validation.

344 in more than deployment, and any of the differences could carry the effect.
345 Here containerisation is held constant and the runtime is varied instead: four
346 container images spanning two OpenSSL series and three V8 major versions,
347 on one machine under one container engine.

348 The pair carrying most of the inference is Node 18 and Node 20: they ship
349 V8 10 and V8 11 respectively while sharing the OpenSSL 3.0 series. In stock
350 images Node and OpenSSL versions otherwise co-vary, which would leave the
351 attribution underdetermined. Because that pairing carries the attribution,
352 the runtime identity of each image — Node, OpenSSL and V8 version, libc,
353 architecture and image digest — is recorded as a datum and cross-checked
354 against the versions each measurement row carried. That check found 0
355 mismatches across the matrix. Section 5.2 states what this design alone
356 cannot separate, and Sections 3.4 and 3.5 describe the two further designs
357 that do.

358 *3.4. Taking the JavaScript engine out*

359 Runtime pairs can separate engine from library only as far as stock
360 builds allow, so the attribution was tested by removing the engine al-
361 together. We read the Node source at the two release lines measured
362 (`src/crypto/crypto_keys.cc` at v18.20.8; `deps/ncrypto` at v22.23.3,
363 which issues the same sequence) and reproduced in C, call for call, what
364 `createPublicKey()` asks of OpenSSL when handed an HMAC secret: three
365 `PEM_bytes_read_bio` scans for a public key, an RSA public key and a
366 certificate, each under an error-queue mark and each followed by `BIO_reset`;
367 then, since none was recognised, `PEM_read_bio_PrivateKey` with Node's
368 passphrase callback; then the C side of Node's error path, which drains
369 the OpenSSL error queue into strings. The V8 side of that error path is
370 deliberately absent; it is what Section 3.5 measures.

371 The program was compiled once per OpenSSL release with one compiler
372 (GCC 13.3.0) and one set of flags, against 8 releases from 3.0.16 to 3.6.1, each
373 built from source with identical configuration and linked by run path so that
374 no system library could be picked up. Each binary reports the version it
375 actually loaded; rows whose loaded version differed from the one requested
376 would be discarded, and there were 0. The protocol is that of Section 3.1:
377 one condition per process, 15 processes per release and condition, 5000 timed
378 calls each, timed with `clock_gettime`, conditions and releases interleaved
379 within rounds. Besides the full sequence we time its parts separately — the

380 three public-key scans, the private-key parse, the error-string drain — and a
381 *successful* parse of an RSA-2048 public key through the same path.

382 3.5. Separating the exception from the parse

383 Inside Node the failed parse is an OpenSSL failure followed by a V8
384 exception: Node constructs an `Error`, attaches `library`, `reason` and `code`
385 properties, throws it through the `createPublicKey()` frame, and the caller
386 discards it. To measure that exception without the parse, a JavaScript
387 benchmark builds the same object — message, property values and error code
388 are captured from a real failure on the runtime being measured, not typed
389 — and throws it from 8 frames below the catch. Variants separate its parts:
390 a plain `Error` thrown and caught in one frame; the Node-shaped error at
391 depth; the same with stack capture disabled; and a Node-internal argument
392 error raised before any OpenSSL call. The real probe is timed alongside,
393 with and without stack capture, so that the stack’s share of it is obtained by
394 paired differencing. All conditions run under official Node release binaries,
395 which bundle their own OpenSSL, on six runtimes — 18.20.8, 20.20.2, 22.23.3,
396 23.11.1, 24.21.0 and 26.10.0 — at 15 processes of 10000 calls per condition.

397 Combining this with Section 3.4 gives a three-way decomposition of the
398 probe on each runtime: the C cost of the OpenSSL release that runtime
399 bundles, the V8 exception, and a remainder we attribute to the binding
400 between them — argument preparation, key-handle allocation and the C++
401 side of error construction. The three come from different processes and cannot
402 be paired, so the interval on the remainder is obtained by resampling each
403 run’s processes independently.

404 3.6. Other libraries

405 To ask whether the defect is one library’s or a design choice’s, we read the
406 key-handling code of six further libraries, chosen as the most used in each
407 ecosystem: `fast-jwt` and `jose` (JavaScript), `PyJWT` (Python), `golang-jwt`
408 (`Go`), and `nimbus-jose-jwt` and `jjwt` (Java). For each we answer one
409 question from source: given a string HMAC secret, does it attempt to parse
410 it as an asymmetric key? We then time HS256 verification in each with the
411 secret in two forms, under the same one-condition-per-process protocol at 15
412 processes of 20000 calls. “String” is the form each library’s documentation
413 shows for an HMAC secret, used the way its interface is meant to be used;
414 “pre-parsed” is the library’s own key object, built once. Where a library
415 does not accept a string we use its rawest accepted form and say so. The

416 JavaScript libraries are run under two runtimes, one bundling OpenSSL 3.0
417 and one bundling 3.5, because the `jsonwebtoken` penalty depends on that.
418 The penalty is the median of per-round paired differences between the two
419 forms.

420 3.7. A second architecture

421 Tables 2 and 4 were measured on one ARM64 host. Both were repeated
422 with unchanged scripts on an x86_64 machine (Intel(R) Xeon(R) Processor @
423 2.80GHz, four virtual CPUs), including the four container images of Table 4;
424 Sections 3.4–3.6 were measured on the same machine. That machine is a
425 virtual machine on shared cloud infrastructure, not the dedicated host a
426 replication of magnitudes would require (Laaber et al., 2019; Leitner and Cito,
427 2016). We use it for what it can support: whether the *ordering* of conditions
428 and runtimes holds on a second instruction set, and ratios between conditions
429 measured interleaved in the same rounds. We do not compare its absolute
430 magnitudes with the ARM64 host’s.

431 3.8. In-situ measurement

432 The deployed service exposes a histogram timing the verification call alone.
433 The instrument is a *paired-scrape difference*: the cumulative histogram is read
434 before and after each arrival-rate step, and the two readings are differenced,
435 which isolates that step’s distribution from the cumulative counters. Means
436 are computed as the difference of the histogram’s `_sum` divided by the difference
437 of its `_count`, so they are exact and do not depend on bucket boundaries;
438 bucket resolution bounds the reported percentiles only.

439 Seven arrival rates are driven, and the whole sweep is run twice on one
440 process: once in ascending order and once in descending order. We report
441 the *range across the two passes*. This is a range of two, not a confidence
442 interval; no standard error is computed from it, and it is not written as a
443 symmetric tolerance. The opposite-order design also serves as a drift test,
444 which Section 6.2 reports.

445 One process and two passes is the whole of the in-situ evidence. It is
446 sufficient to establish that the cost is present in deployment and of what order;
447 it is not sufficient to characterise the shape of the cost-versus-rate relation,
448 and Section 6.1 is written accordingly.

449 *3.9. Prevalence: three instruments, three frames*

450 Prevalence is approached with three instruments that fail in different ways.
451 Their sampling frames differ and are not interchangeable, so no population
452 proportion is computed from them. All three were captured on 2026-09-18.

453 *Population co-occurrence counts.* Code-search counts over public repositories,
454 in four cells: two languages by two import forms. The denominator of a
455 cell is files importing the library; the numerator is files that *also mention* the
456 pre-parsing interface. This measures co-occurrence of two strings in one file,
457 not verified usage. The denominators are returned rounded — each is an exact
458 multiple of 1024 — so they are magnitudes rather than counts established
459 file by file.

460 *A classified sample of call sites.* Call sites located by search, deduplicated
461 to one file per repository so that no project dominates, with the key argument
462 traced within its file. This is a convenience sample from a relevance-ranked,
463 undocumented ranking; inclusion probabilities are unknown and no weighting
464 back to the population is possible.

465 *A targeted counter-search.* Every file co-occurring the library with the
466 pre-parsing interface, fetched in full and adjudicated by hand. This frame
467 is deliberately biased toward finding the pattern — that is its purpose as a
468 negative control — and it is therefore disqualified as a prevalence estimator.

469 Three reasons preclude a population-level prevalence. The population
470 numerator and the verified numerator measure different things: of 13 auto-
471 matically flagged call sites, 5 were false positives on hand inspection, so the
472 co-occurrence signal carries an error rate that would be inherited, unmeasured,
473 by any extrapolation. The sample has no inclusion probabilities. And the
474 counter-search is selected on the outcome. Results are therefore reported
475 *in our sample*, or *among repositories that co-occur the interface and call the*
476 *validator*, and never as a population proportion. This constraint is carried
477 into the abstract and the conclusion.

478 *3.10. Host state is recorded per measurement*

479 A service measured on a host it shares with its own load generator,
480 ingress and sidecars is measured together with whatever else that host is
481 doing. We observed this directly while building the instrument: re-running
482 one configuration with a background indexing process consuming host CPU
483 moved the point at which throughput collapsed by hundreds of requests per
484 second, and both runs are committed in the replication package (Katha et al.,
485 2026). A curve that moves with unrelated host activity does not characterise

the service, and this paper therefore makes no capacity claim. Every in-situ window instead carries the host load average observed during it, committed beside the measurement, and Section 6.3 uses it.

4. RQ1: Where the cost goes

4.1. The decomposition

Table 2 gives each operation in the validation path, timed in isolation on the host under the protocol of Section 3, and Figure 3 plots it.

Table 2: Cost of each operation in the validation path. Medians of per-process medians over 15 processes of 40000 calls; 95% percentile bootstrap intervals over processes, which with 15 units are indicative rather than calibrated (Section 3.1).

Operation	Median (μs)	95% interval (μs)
<code>verify()</code> , string secret	29.92	[29.75, 30.12]
discarded asym. parse	18.04	[17.83, 18.25]
successful asym. parse	13.83	[13.62, 13.92]
<code>verify()</code> , pre-parsed key	6.042	[5.958, 6.125]
HMAC, string key	2.208	[2.208, 2.250]
HMAC, pre-parsed key	2.166	[2.125, 2.167]
structural decode	1.292	[1.250, 1.292]
symmetric key constructor	0.583	[0.542, 0.583]
timer overhead	0.042	[0.042, 0.042]

The two emphasised rows are the result. The signature primitive costs 2.208 μs of a 29.92 μs call. The conversion to which the remainder is most readily attributed costs 0.583 μs . The discarded attempt that precedes that conversion costs 18.04 μs , which is 60% of the whole call and its largest single component.

Supplying a pre-parsed key instead of a string reduces the call by 23.92 μs [23.58, 24.00] over 15 paired rounds.

The natural repair is also wrong. Once the signature primitive is shown to be small, one explanation for the remainder suggests itself immediately: that the cost lies in converting key material from a string into a key object on every call. It is the natural explanation, and Table 2 refutes it. That conversion

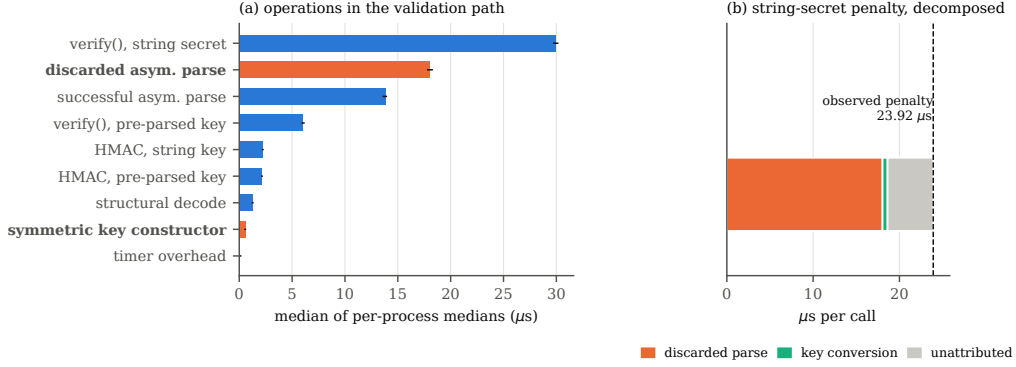


Figure 3: (a) Every operation in Table 2, on a linear scale, with the two components the paper turns on emphasised; whiskers are the 95% percentile bootstrap intervals over 15 processes. (b) The 23.92 μs penalty for supplying a string secret rather than a pre-parsed key, decomposed. The three segments are medians of per-round paired differences, not a partition of a single measurement, and they do not sum to the penalty: the 5.292 μs remainder is unattributed (Section 4.3).

costs 0.583 μs , so the attribution is wrong by roughly the ratio between that figure and 18.04 μs .

A second observation is usually offered in support of the conversion account: an asymmetry between HMAC and RSA costs, read as conversion being dearer than parsing a PEM. It is backwards. A successful asymmetric parse costs 13.83 μs against 0.583 μs for the conversion. The real asymmetry is that the RSA path’s parse *succeeds*, making its cost useful work, while the HMAC path’s fails, making it waste.

4.2. What the 18.04 μs is made of

The 18.04 μs is the cost of the discarded attempt taken whole. It contains at least two things: the work OpenSSL performs before concluding that the input is not an asymmetric key, and the construction and unwinding of the exception that returns that conclusion to the library. Table 2 already bounds the split: a *successful* asymmetric parse — the same entry point, reaching a conclusion and throwing nothing — costs 13.83 μs , the same order as the failing one, so the exception cannot be most of it. Table 3 replaces the bound with a decomposition, combining the C measurement of the same OpenSSL release (Section 3.4) with the exception baseline (Section 3.5) on six official runtimes.

Table 3: The discarded attempt decomposed, per runtime, in μs (x86_64). *Probe*: `createPublicKey(secret)` inside Node, thrown and caught. *C parse*: the same OpenSSL calls issued from C against the OpenSSL release that runtime bundles, built from source. *V8 exception*: an `Error` of the same shape, message and properties thrown from 8 frames and discarded, with no OpenSSL call. *Binding*: the probe less the other two; its interval, from independent resampling of the three runs, is in the replication package. Medians of per-process medians over 15 processes (probe, exception) and 15 processes (C). The C column uses a stock build of each release, not Node’s own build of it, so the binding column is approximate (Section 10).

Runtime	OpenSSL	Probe	C parse	V8 exception	Binding
Node 18.20.8	3.0.16	390.5	394.4	4.781	-8.694
Node 20.20.2	3.0.19	443.4	431.3	4.796	7.292
Node 23.11.1	3.0.16	383.5	394.4	3.959	-14.88
Node 22.23.3	3.5.8	18.98	8.447	4.021	6.515
Node 24.21.0	3.5.8	19.54	8.447	4.136	6.952
Node 26.10.0	3.5.8	18.01	8.447	3.421	6.145

523 Three things follow. First, on every runtime bundling OpenSSL 3.0 the C
524 parse alone matches the probe to within a few percent — 101% of it under
525 Node 18.20.8 and 103% under Node 23.11.1, the excess over 100% being the
526 difference between our build of the release and Node’s. There the discarded
527 attempt *is* the OpenSSL parse, and it would cost the same with no JavaScript
528 engine present. Second, the exception is small and nearly constant across
529 runtimes: the Node-shaped error costs 4.781 μs under Node 18.20.8 and
530 3.421 μs under Node 26.10.0 (1.2% and 19.0% of the respective probes), and
531 roughly half of it is stack capture (with capture disabled, 2.204 and 1.930 μs).
532 Disabling stack capture for the real probe saves 5.397 μs and 1.767 μs on
533 the same two runtimes, the same order as the synthetic figure. Third, on
534 runtimes bundling OpenSSL 3.5, where the parse is cheap, the three parts
535 are of comparable size: under Node 26.10.0 the parse is 47% of the probe,
536 the exception 19.0% and the binding 34.1%.

537 This is why the component is named a discarded *parse attempt* rather
538 than a discarded exception. Where it is expensive the exception is about a
539 hundredth of it, and even where it is cheap the exception is a fifth. These
540 runtimes are Linux builds on x86_64; the ARM64 host figure of Table 2 was
541 not itself decomposed this way, and its 18.04 μs is of the same order as the
542 OpenSSL 3.5 rows here, not the OpenSSL 3.0 rows.

543 *4.3. The parts do not account for all of it*

544 If the penalty for supplying a string were exactly the discarded attempt
 545 plus the conversion, the two would sum to it. They do not, quite:

	observed penalty	23.92 μs
546	discarded parse + conversion	18.62 μs
	residual	5.292 μs

547 All three quantities are medians of per-round paired differences, per
 548 Section 3.1. The residual’s interval, [4.916, 5.501], excludes zero. The two
 549 measured parts therefore account for 77.9% and not more; the remainder
 550 lies in the library’s own type and claim checking along the non-pre-parsed
 551 path. We report it rather than absorbing it into the headline, because a
 552 decomposition whose parts are declared to sum exactly to the whole has
 553 usually not been checked.

554 The pre-parsed path carries a smaller unexplained remainder of the same
 555 kind, and we record it for consistency rather than leaving the asymmetry
 556 unremarked: `verify()` with a pre-parsed key costs 6.042 μs against 1.292 μs of
 557 structural decode plus 2.166 μs of HMAC, leaving roughly 2.58 μs unattributed.
 558 Neither residual was measured directly; both are the subject of conditions we
 559 did not run, and both are stated as unattributed rather than assigned.

560 *4.4. Confirmation without differencing*

561 The decomposition reaches its conclusion by subtracting conditions — the
 562 same inference style this paper criticises elsewhere. It is therefore checked
 563 against V8’s sampling profiler, which attributes self time to named frames
 564 with no arithmetic on our part. At 50000 iterations per run and 3 runs per
 565 environment, matched so the percentages are comparable, the asymmetric key
 566 parse accounts for a median 66.2% of self time on the host (range 65.3–67.4
 567 across runs), 95.3% (95.1–95.6) under Node 18, and 61.5% (58.5–61.6) under
 568 Node 26. The frame does not appear among the top frames of the pre-parsed
 569 path in any environment.

570 Two methods, one answer. We rely on the ordering and the dominance
 571 rather than on the exact percentages: sampling attribution depends on rate
 572 and on what else the process is doing.

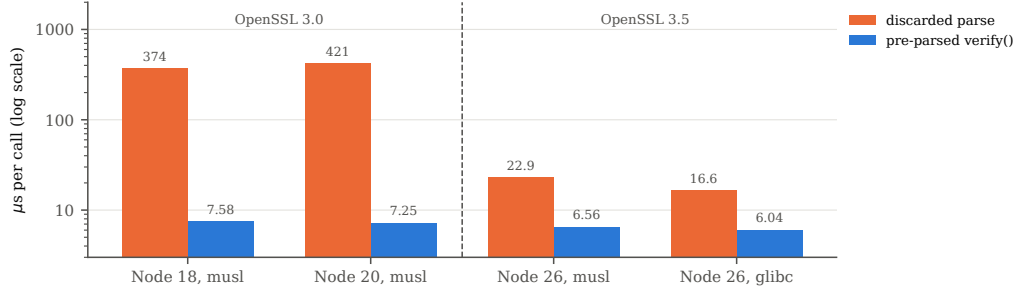


Figure 4: The four container rows of Table 4 on a logarithmic axis. The discarded parse spans $25.36\times$ across images while the pre-parsed call spans $1.26\times$. The dashed rule marks the OpenSSL series boundary. Within this matrix it is not a controlled boundary, because the two fast images also carry a newer engine and a different build (Section 5.2); Section 5.5 controls it. The uncontainerised host row is omitted because its comparison with the container rows is confounded.

5. RQ2: What governs the magnitude

Table 4 varies the runtime with containerisation held constant, and Figure 4 plots the two columns that matter.

Table 4: The same measurement across four container runtimes on one machine, 10 processes each of 20000 calls. The *Disc. parse*: the discarded asymmetric parse; *Pre-parsed*: `verify()` with a pre-parsed key. [‡]The host row is from Table 2 and is not containerised. [†]Inferred from the run timestamp, not recorded per row: that run predates the per-measurement OpenSSL field (Section 10).

Runtime	OpenSSL	V8	Disc. parse (μs)	Pre-parsed (μs)
Node 18 musl	3.0.16	10	373.79	7.58
Node 20 musl	3.0.19	11	420.62	7.25
Node 26 musl	3.5.7	14	22.94	6.56
Node 26 glibc	3.5.7	14	16.58	6.04
host [‡]	3.6.3 [†]	14	18.04	6.042

5.1. Engine major version does not distinguish the two slow runtimes

Node 18 and Node 20 ship V8 10 and 11 — different major versions of the engine — and share the OpenSSL 3.0 series. Both pay of order 373.79–420.62 μs . Node 26 ships OpenSSL 3.5 and pays 22.94 μs on the same libc. Within the OpenSSL 3.0 series, then, changing the engine’s major version

581 does not move the quantity; had Node 20 been fast, the attribution would
582 have been to the engine.

583 *5.2. What this design cannot separate*

584 That argument establishes a negative within one OpenSSL series. It does
585 not establish the positive attribution for the change between series, and we
586 state the limitation rather than leaving it implicit.

587 The fast rows are Node 26. They carry OpenSSL 3.5 *and* V8 14 *and* every
588 other change in that build. OpenSSL version is therefore perfectly confounded
589 with engine version and with build configuration across the 3.0–3.5 boundary.
590 Nothing in this matrix can separate them. The absence of variation between
591 V8 10 and V8 11 does not license the inference that V8 14 is also irrelevant;
592 that is a different comparison, and we did not make it.

593 What the matrix supports on its own is the following, and no more: the
594 magnitude co-varies with the bundled OpenSSL version across four images;
595 the two slow images differ in engine major version and do not differ in
596 magnitude; and the two libc variants of one image differ by less than the
597 difference between series. Separating OpenSSL from the engine requires a
598 crossed design — a runtime pairing a newer engine with an older OpenSSL,
599 or the parse measured outside Node entirely. Section 5.5 reports both, and it
600 is on them, not on this matrix, that the causal wording used in this paper
601 rests.

602 *5.3. The effect is confined to key parsing*

603 Across the four container rows, pre-parsed validation spans $1.26\times$, the
604 HMAC primitive $1.39\times$ and structural decode $1.35\times$. The discarded parse
605 spans $25.36\times$. The successful parse moves with it ($11.74\times$), so this is the cost
606 of key parsing in general rather than anything peculiar to the failure path.
607 The failure is simply the case in which an application pays that cost on every
608 request for a result it discards.

609 *5.4. What this design does not isolate*

610 A reader may notice that the uncontainerised host row is of the same
611 order as the Node 26 container row. That comparison is confounded and we
612 draw no conclusion about containerisation from it: the host runs a different
613 operating system from the containers, so operating system, C library and Node
614 build all change alongside the container boundary. Isolating containerisation
615 would require a Linux-native baseline, which we did not obtain. The claims

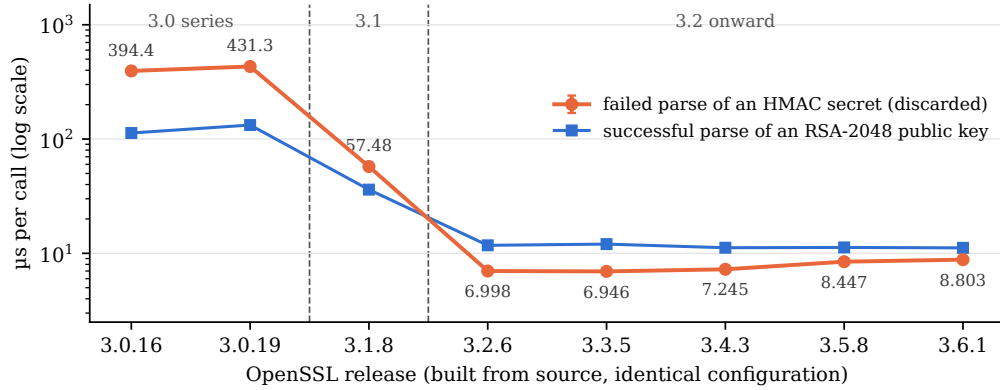


Figure 5: The failed parse of an HMAC secret issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one configuration (Table 5, column *Full*), beside a successful parse of an RSA-2048 public key through the same path. Whiskers are 95% percentile bootstrap intervals over 15 processes and are narrower than the markers. The cost falls in two steps, at 3.1 and 3.2, and is flat thereafter.

above concern only the four container rows, among which containerisation is constant.

5.5. Caused by OpenSSL: the parse without an engine

Table 5 removes Node entirely: the OpenSSL calls `createPublicKey()` makes for an HMAC secret, issued from C against 8 OpenSSL releases built identically, and Figure 5 plots its first and last columns.

In plain C the failure costs 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8, a ratio of 46.69 [46.16, 46.90]. Nothing else differs between those two measurements: the program, the compiler, the flags, the machine and the protocol are the same. The magnitude is therefore *caused* by the OpenSSL release. That is a claim about this code path in these releases; it says nothing about OpenSSL performance generally.

The table also says where in OpenSSL the cost lives and when it went away. Almost all of it is the private-key parse — 99% of the full sequence on 3.0.16 — the step at which OpenSSL 3 hands the input to its provider-based decoders; the three public-key scans reject the input at the PEM header and are cheap in every release, and the error strings cost under a microsecond. The fall came in two steps: to 57.48 μ s in 3.1.8, and to 6.998 μ s in 3.2.6, after which it is flat to within a few microseconds through 3.6.1. We measured the

Table 5: The failed asymmetric parse issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one compiler and one set of flags (x86_64), in μ s. Medians of per-process medians over 15 processes of 5000 calls. *Full*: everything `createPublicKey()` asks of OpenSSL for an HMAC secret, including the error-string drain; 95% intervals for it are in the replication package and are within 0.9% of the median for every release. Its parts follow. *Successful parse*: an RSA-2048 public key through the same path, for comparison.

OpenSSL	Full	Private-key parse	Public-key scans	Error strings	Successful parse
3.0.16	394.4	390.9	2.287	0.796	112.9
3.0.19	431.3	424.0	2.231	0.869	132.6
3.1.8	57.48	54.84	2.196	0.497	36.09
3.2.6	6.998	4.487	2.057	0.459	11.75
3.3.5	6.946	4.465	2.074	0.474	12.04
3.4.3	7.245	4.554	2.098	0.500	11.19
3.5.8	8.447	5.757	2.051	0.483	11.26
3.6.1	8.803	5.985	2.234	0.472	11.16

steps, not the source changes that produced them; identifying those would take a bisection of the OpenSSL history that we have not done. A successful parse of real key material fell with it, which is why the effect is confined to key parsing in general rather than to the failure path.

Engine and library crossed. The official runtimes of Section 3.5 contain the pairing stock container images lack: Node 22.23.3 ships V8 12.4.254.21 with OpenSSL 3.5.8, while Node 23.11.1 ships the *newer* V8 12.9.202.28 with the *older* OpenSSL 3.0.16. The probe costs 18.98 μ s on the first and 383.5 μ s on the second. The Node 18/20 pair of Table 4 showed that a change of engine at a fixed OpenSSL series leaves the cost where it was; this pair shows that a newer engine does not rescue an older OpenSSL. Together with the C measurement, the engine is excluded from both directions.

5.6. A second architecture

Table 6 repeats the runtime matrix, with the same images and scripts, on x86_64 (Section 3.7). Absolute values are not compared across the two machines; the question is whether the ordering holds.

The property the attribution rests on — every OpenSSL 3.0 image slower than every OpenSSL 3.5 image — holds on both architectures. The strict

Table 6: The runtime matrix of Table 4 on two architectures, same images and scripts, in μs . ARM64: 10 processes of 20000 calls. x86_64, a shared cloud virtual machine: 10 processes of 20000 calls. Absolute values are not comparable across the two machines.

Image	OpenSSL	Discarded parse		Pre-parsed <code>verify()</code>	
		ARM64	x86_64	ARM64	x86_64
<code>node:18.20.8-alpine</code>	3.0.16	373.79	607.1	7.58	12.66
<code>node:20-alpine</code>	3.0.19	420.62	556.3	7.25	11.89
<code>node:26.6.0-alpine</code>	3.5.7	22.94	42.13	6.56	10.94
<code>node:26.6.0-bookworm</code>	3.5.7	16.58	19.91	6.04	9.701

rank order does not: the two OpenSSL 3.0 images, Node 18 and Node 20, exchange places between ARM64 and x86_64. Their difference is small against the gap between series on either machine, and no claim in this paper depends on their relative order. On x86_64 the discarded parse spans $30.50\times$ across the images while pre-parsed validation spans $1.31\times$, against $25.36\times$ and $1.26\times$ on ARM64. The host decomposition of Table 2 was repeated too (Node 22.22.2, OpenSSL 3.5.5): the discarded parse is $19.42 \mu s$ of a $36.48 \mu s$ call (53%), the symmetric constructor $1.261 \mu s$ and the HMAC primitive $3.105 \mu s$, so the component ordering of Table 2 holds there as well. The full x86_64 host table is in the replication package.

6. RQ3: Deployment and prevalence

6.1. In-situ cost across arrival rates

The deployed service was driven at seven arrival rates, each measured by the paired-scrape difference of Section 3.8, with the whole sweep run ascending and then descending on one process. Table 7 reports the range across the two passes at each rate for the configuration the service deploys, a string secret, and Figure 6 plots it.

One observation follows, and it is the one this section claims: the cost is present in deployment and is far larger than the isolated host figure of Table 2, consistent with Section 5 in that the deployed runtime is the one carrying the older OpenSSL.

Two things visible in Table 7 need care, and we set them out explicitly.

Wall-clock exceeds the prediction; processor time does not. The deployed runtime’s discarded parse costs $373.79 \mu s$ (Table 4), and the remainder of the

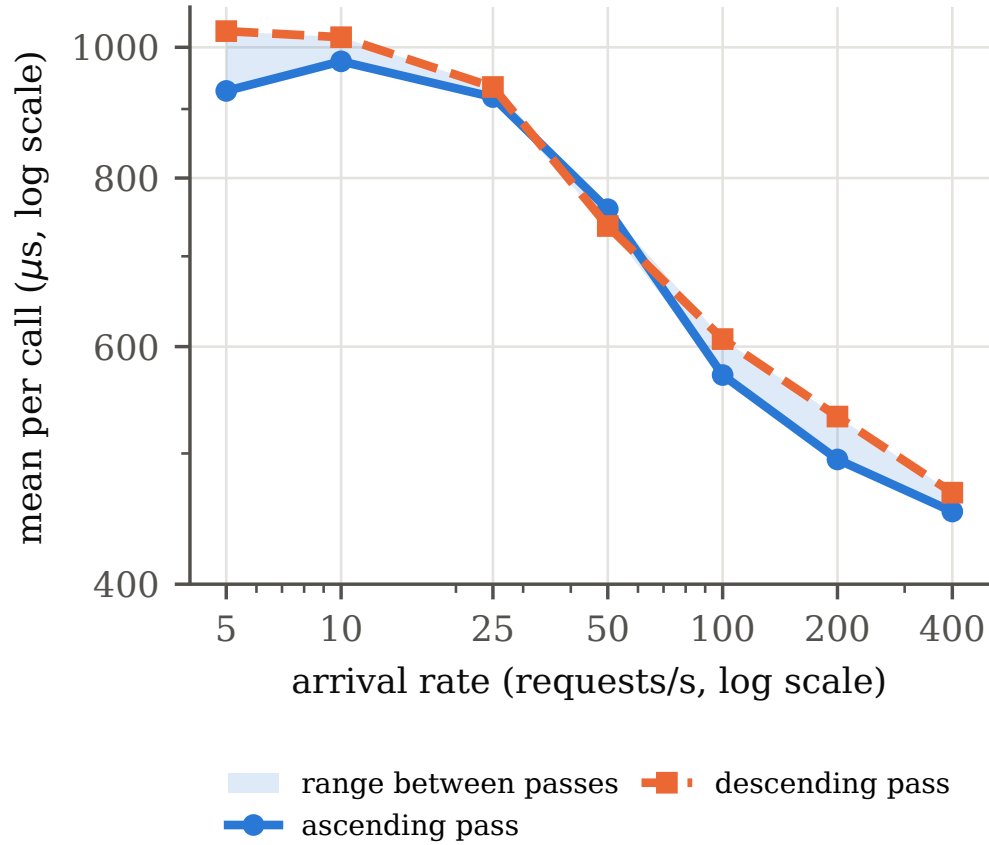


Figure 6: In-situ validation cost against arrival rate, string secret. The two passes are drawn separately rather than averaged: Section 6.2 reports that they differ systematically, and averaging would conceal that. The shaded band is the range between the passes, which is two observations rather than an interval.

Table 7: In-situ validation cost with a string secret, by arrival rate. The two columns are the ascending and the descending pass over the same seven rates on one process; each is a single window, and the pair is two observations rather than an interval. Host load average during each window is committed with the measurement.

Arrival rate (requests/s)	Mean per call (μ s)	
	ascending	descending
5	928.24	1027.97
10	976.28	1017.34
25	918.54	934.47
50	758.57	737.12
100	571.59	607.78
200	494.91	532.50
400	452.82	467.62

validation path is of order 7.58 μ s, giving an expected total near 381 μ s. The wall-clock means observed across the sweep are 453–1028 μ s, exceeding that by 71–647 μ s. Contention inflates a duration rather than deflating it, so these are upper bounds, and Section 6.4 shows where the excess is not: measured as processor time, the validation call costs within 0.8% of the prediction. The excess is therefore time the call spent off the processor on a shared host, not additional work. We did not decompose that off-processor time further, and no claim below depends on it.

The decline with arrival rate is not interpreted. Cost falls monotonically from 928.24 μ s at 5 requests/s to 452.82 μ s at 400 requests/s. This is the opposite of what contention alone would produce, and several mechanisms unrelated to validation would produce it — a fixed per-window overhead amortising over more calls, processor frequency scaling and idle-state exit latency at low rates, cache and branch-predictor residency, or continued tier-up in the engine. We did not run the conditions that would distinguish them. We therefore report the numbers and do not claim that validation cost depends on arrival rate; establishing that would require varying window duration at fixed rate, pinning the frequency governor, and extending warmup at low rates, none of which we did.

696 6.2. *The two passes differ systematically*

697 Sweeping up and then down is not only replication; it detects warm-up,
698 thermal and hysteresis effects. If the difference between passes were noise,
699 its sign would alternate across rate points. It does not. In the string-secret
700 sweep the descending pass is slower at 6 of 7 rate points; in the pre-parsed
701 sweep the descending pass is *faster* at 7 of 7. A consistent sign across seven
702 points is not what noise produces.

703 This has a direct consequence for how the ranges in Table 7 should be read.
704 A systematic component means the two passes are not independent samples of
705 a common value, so the range between them understates uncertainty further
706 than a range of two already does. We therefore report ranges and draw no
707 inference that depends on their width. It is also a methodological finding in
708 its own right: a single-direction sweep would have produced a curve carrying
709 a consistent bias with no way to detect it.

710 6.3. *Host load, not the condition, drives the outliers*

711 Agreement between passes varies considerably across windows. We dis-
712 tinguish *relative* disagreement, the gap between passes as a fraction of their
713 mean, from *absolute* disagreement, the gap in microseconds; the two behave
714 differently and the distinction carries this subsection.

715 The widest relative disagreement in the dataset and the tightest occur at
716 the same arrival rate in different sweeps. The wide one is the highest-load
717 window in its own sweep on the descending pass, though not on the ascending
718 pass, whose maximum load falls at a different rate. The tight one, at the
719 same arrival rate in another sweep, sits at a lower load on both passes and
720 its two passes agree to within $0.02\ \mu\text{s}$. Relative disagreement therefore does
721 not track the condition under measurement.

722 We considered two alternative explanations for the relative asymmetry and
723 reject both. A small-mean effect would require absolute disagreement to be
724 roughly constant, so that its relative size grew only as means shrank; absolute
725 disagreement instead varies over three orders of magnitude and is largest
726 where the means are largest, which is the opposite pattern. A measurement
727 floor would require the affected values to sit at the instrument’s resolution;
728 they do not, and in any case the means reported here are computed from the
729 histogram’s sum and count rather than from its buckets, so bucket resolution
730 cannot bound them.

731 The operative consequence is that in-situ figures are reported with the
 732 host load recorded during their window, and comparisons between windows
 733 at different loads are not treated as comparisons between conditions.

734 *6.4. Corroboration: the cost is processor time, and it matches the prediction*

735 An in-situ figure of this magnitude has two readings. Either it is work the
 736 service performs, or it is time the request spends waiting — queued behind
 737 other requests, descheduled by a contended host, or inflated by the instrument.
 738 We separate them with an A/B of the same service at one arrival rate, with
 739 validation enabled in one window and disabled in the next, on the same
 740 pod running the same image, and read the processor time the application
 741 container was charged from its cgroup accounting rather than from a clock.
 742 Table 8 reports it.

Table 8: Validation enabled against validation disabled, at one arrival rate, two 180 s windows on the same pod and image, 36001 requests each. A signed token is on the wire in both arms; the disabled arm removes only the handler’s `jwt.verify()` call. Application and sidecar CPU are means of one-second cgroup samples in the window; CPU per call is that mean divided by the achieved rate. Event-loop lag is the mean of the per-scrape p99. Host load is a single sample at window start.

	disabled	enabled
Achieved rate (req/s)	200.006	200.006
Application CPU (millicores)	69.45	146.38
CPU per call (μs)	347.3	731.9
Sidecar CPU (millicores)	53.40	49.13
Event-loop lag (ms)	3.137	3.159
Client mean latency (ms)	1.6802	2.0332
Host load, 1 min	3.53	5.11

743 Enabling validation adds 384.6 μ s of processor time per call. The runtime
 744 matrix predicts 381 μ s for the same work on the same runtime (Section 6.1);
 745 the two agree to within 0.8%. They are obtained by instruments that share
 746 no code: cgroup CPU accounting on a deployed pod, and a single-process
 747 microbenchmark in an isolated container. A request that queues or is de-
 748 scheduled accrues wall-clock time but no CPU, so this figure cannot contain
 749 queueing delay.

750 Three observations weigh against reading the difference as an artefact of
 751 the two windows. The sidecar, in the same pod under the same contention,
 752 did not rise with the application: 53.40 millicores disabled against 49.13

753 enabled. The one-second CPU samples of the two arms do not overlap at
754 all — disabled never exceeded 100.577 millicores in 127 samples, enabled
755 never fell below 114.963 in 127 — which separates the arms without a
756 distributional assumption. And event-loop lag is unchanged, so neither arm
757 was near saturation. The two windows are two observations, not a sample:
758 the difference carries no interval and we compute none.

759 *6.5. How common is the avoiding pattern, in our sample?*

760 Three instruments, three frames, reported separately as Section 3.9 re-
761 quires. All counts were captured on 2026-09-18. None of the figures below is
762 a population proportion.

763 *Population co-occurrence.* Of 790528 public JavaScript files importing the
764 library through `require`, 29 also mention the pre-parsing interface; for
765 JavaScript through `import` the figures are 524288 and 38; for TypeScript
766 through `require`, 8288 and 0; and for TypeScript through `import`, 333824
767 and 101. Cells are not pooled into a rate: a file using both import forms
768 would be counted twice, so their sum 1656928 is an upper bound on the union
769 and the largest cell 790528 a lower bound. No cell exceeds 0.0303% of files
770 mentioning the interface. These are search-engine estimates of file counts over
771 public repositories only, they measure co-occurrence rather than use, and the
772 denominators are returned *rounded* — each is an exact multiple of 1024 — so
773 they should be read as magnitudes rather than as counts established file by
774 file.

775 *Classified call sites.* Of 327 call sites classified across 247 distinct repositories
776 in our sample, 0 pass a pre-parsed key. Environment variables account for
777 176 of those call sites, untraced identifiers and member expressions 93, string
778 literals 52, file contents 4 and buffers 2. No file in this corpus contains the
779 pre-parsing interface at all, in any spelling, so the zero reflects the pattern's
780 absence from the sample rather than a classifier that cannot see it. Incidentally,
781 309 of 327 call sites (94%) pass no algorithm restriction, which Section 8.4
782 returns to.

783 Matching only the conventional `jwt.verify()` spelling would filter the
784 corpus toward one import idiom, so call sites are found by resolving the
785 local binding of the import — covering arbitrary aliases, namespace imports,
786 destructured and renamed imports, and direct calls on the `require` expres-
787 sion. That binding resolution contributes 12 of the call sites above that the
788 conventional spelling misses, of which 0 pass a pre-parsed key.

789 *Targeted counter-search.* A sample can only show a pattern is rare if it
790 could have found it. Every file co-occurring the library with the pre-parsing
791 interface was therefore fetched: 191 files, of which 120 contain a validation
792 call, spanning 112 distinct repositories. An automated pass flagged 13 call
793 sites as passing a pre-parsed key and all were then read by hand: 5 are false
794 positives — projects defining their own helper function of the same name,
795 one of which returns a string — 1 could not be resolved within its file, and 7
796 are genuine, across 6 distinct projects.

797 *The rate, with its denominator.* Among the 112 repositories that both refer-
798 ence the pre-parsing interface and call the validator, 6 use it: 5.36%. That
799 denominator is the most favourable available, since those projects demonstra-
800 bly know the interface exists, and it is not the population. Within this frame
801 the pattern is rare rather than absent, and it is rare even among projects
802 already aware of it. We do not extrapolate beyond the frame.

803 7. RQ4: One library, or a design choice?

804 A cost found in one library is an observation about that library. It becomes
805 a statement about a class of defect only if the mechanism can be named
806 independently of the library and predicts which other libraries have it. The
807 mechanism here can be named: *resolving key material of unknown type by*
808 *attempting the asymmetric parse first, and treating its failure as the signal*
809 *that the material is a shared secret.* Table 9 tests the prediction against six
810 further libraries.

811 *What the source says.* Only `jsonwebtoken` attempts the asymmetric parse.
812 The others resolve the ambiguity in one of three ways, none of which involves
813 a parse that can fail on a shared secret. *Inspect, then dispatch:* `fast-jwt`
814 examines the text of the key — whether it carries a PEM header or is a JSON
815 Web Key — and calls exactly one constructor, skipping even that inspection
816 when every permitted algorithm is HMAC; `PyJWT` likewise inspects the text,
817 and rejects PEM- or SSH-looking material offered as an HMAC key rather
818 than trying to parse it. *Refuse the ambiguous input:* `jose` does not accept
819 a string for an HMAC key, and `golang-jwt` accepts only a byte slice for
820 HMAC verification, returning a key-type error for anything else. *Type the*
821 *key:* `jjwt` requires a `SecretKey` for HMAC and a `PublicKey` for signatures,
822 so the distinction is made by the Java type system; `nimbus-jose-jwt` offers
823 a constructor that takes a string and converts its bytes without parsing them.

Table 9: HS256 verification across seven JWT libraries in four languages (x86_64), in μs . *Asym. first?*: given a string HMAC secret, does the library attempt to parse it as an asymmetric key, as read from its source. *String* and *pre-parsed*: the secret in the form the library’s documentation shows, and in the library’s own key object built once (Section 3.6). *Penalty*: median of per-round paired differences, with a 95% bootstrap interval over rounds. Medians of per-process medians over 15 processes of 20000 calls. Absolute costs measure different amounts of work in different runtimes and are not a ranking of libraries. ^aNo string accepted for an HMAC key; the rawest accepted form (bytes) is used. ^bA string is accepted only by a deprecated setter that resolves it once, when the parser is built.

Library	Lang.	Version	Crypto backend	Asym. first?	String	Pre-parsed	Penalty	95% interval
jsonwebtoken	JS	9.0.3	OpenSSL 3.0.19	yes	476.2	10.23	465.9	[465.2, 467.8]
jsonwebtoken	JS	9.0.3	OpenSSL 3.5.8	yes	34.22	9.844	24.56	[24.07, 24.82]
fast-jwt	JS	6.3.3	OpenSSL 3.0.19	no	7.564	7.570	0.070	[-0.116, 0.111]
fast-jwt	JS	6.3.3	OpenSSL 3.5.8	no	7.917	7.934	-0.068	[-0.134, 0.197]
jose	JS	6.2.12	OpenSSL 3.0.19	no ^a	121.2	78.42	42.66	[38.50, 45.89]
jose	JS	6.2.12	OpenSSL 3.5.8	no ^a	98.89	73.98	24.80	[21.91, 26.51]
PyJWT	Python	2.15.0	hashlib, OpenSSL 3.0.13	no	41.33	41.28	-0.103	[-0.457, 0.338]
golang-jwt	Go	5.3.1	Go crypto/hmac	no ^a	4.810	4.764	0.034	[-0.042, 0.117]
nimbus-jose-jwt	Java	10.10	JCA SunJCE	no	4.358	4.229	0.304	[-0.098, 1.016]
jjwt	Java	0.13.0	JCA SunJCE	no ^b	12.15	12.33	-0.406	[-0.715, 0.648]

824 Refusing a string does not by itself remove per-call work, however: `jose`,
825 handed raw bytes, imports them into a WebCrypto key on every call.

826 *What the measurements say.* Of the seven libraries, two pay for being handed
827 their raw form rather than a key object, in the sense that the paired interval
828 on the penalty excludes zero; for the other five it does not, and their point
829 estimates are within a microsecond of zero. The two are `jsonwebtoken`
830 and `jose`, and they pay for different reasons. `jsonwebtoken` performs the
831 failing asymmetric parse: its penalty is 465.9 μs under OpenSSL 3.0.19 and
832 24.56 μs under OpenSSL 3.5.8, a string-to-pre-parsed ratio of 46.55 and 3.48
833 respectively. `jose` parses nothing that can fail, but imports the raw bytes
834 into a WebCrypto key on every call: 42.66 μs and 24.80 μs under the same
835 two runtimes. Both penalties grow on OpenSSL 3.0, so the OpenSSL release
836 sets the price of per-call key handling generally; but the failing parse is the
837 costliest form of it, about 11 times `jose`’s on OpenSSL 3.0. Two of the
838 libraries that pay nothing resolve the key once, when a verifier or parser is
839 built (`fast-jwt`, `jjwt`), so a raw secret costs nothing per call by construction;
840 that is itself the design point.

841 *What this establishes.* The libraries without the problem matter as much as
842 the ones with it. All seven validate the same kind of token with the same

primitive — against the same OpenSSL, in the JavaScript cases — and five pay nothing for a raw secret. The cost therefore follows a design choice, resolving key material on every call, and not JSON Web Token validation, HMAC or OpenSSL as such: OpenSSL 3.0 sets the price, but only a library that makes that choice is charged it, and one that makes it by way of a failing parse is charged the most. The failing parse is exactly what the order-of-evaluations profiler of Selakovic et al. (2017) is designed to find, and the alternatives above are the fixes that line of work suggests: reorder the test, make it cheap, or move it off the per-call path. The comparison does not estimate how many libraries make the choice; it shows that making it is what matters.

8. RQ5: The cost as a security property

Sections 4–7 treat the discarded attempt as a cost. This section asks who can cause it, why it is there, and what removing it does to the library’s defences. Its evidence is the library’s source at the releases named, the library’s own test suite, and the measurements already reported; it introduces no new timing condition, and where a statement rests on reading code rather than on measurement we say so.

8.1. Threat model

The adversary is a remote client that can place arbitrary bytes where the service expects a bearer token, including every field of the token header, and knows which library the service uses. It does not know the shared secret and cannot produce a valid HMAC. The service verifies HS256 tokens with a shared secret supplied in the form the library’s documentation shows, a string — the form every call site in our sample uses (Section 6.5). The asset of interest is key-type separation: a token declaring an HMAC algorithm must never be verified against asymmetric key material, nor the reverse (Sheffer et al., 2020). Out of scope are compromise of the secret, defects elsewhere in the library, and availability. We did not drive adversarial traffic at the deployed service and make no availability or capacity claim; the question here is which inputs reach the discarded attempt, not how many a service can absorb.

8.2. The attempt precedes authentication

Figure 7 shows the order in which `verify()` performs its checks at tag v9.0.3. Reaching the key-resolution block of Figure 2 requires only

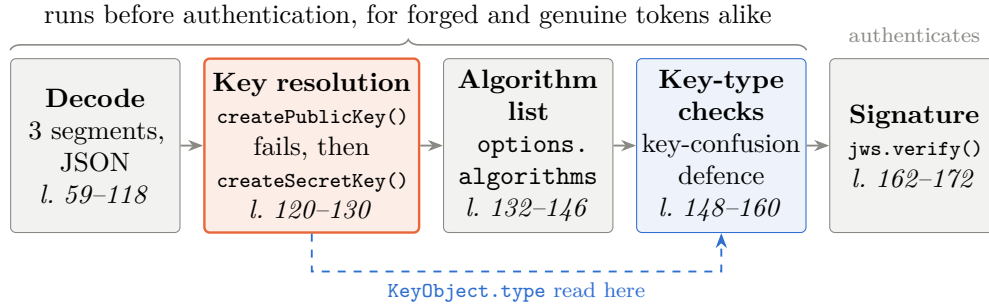


Figure 7: Order of operations in `verify()` at tag v9.0.3, with the file’s line numbers. Every step up to and including the algorithm and key-type checks runs before the signature is verified, and so runs for a forged token exactly as for a genuine one. The discarded attempt sits inside key resolution; the key-type checks that implement the library’s key-confusion defence read the type of the key object resolution produces (dashed arrow). Pinning permitted algorithms is enforced at the third step and therefore does not prevent the second.

877 that the token have three dot-separated segments, a header and payload that
 878 decode as JSON, and a non-empty signature segment (lines 59–118). Nothing
 879 on that path depends on the secret. The permitted-algorithm list is consulted
 880 afterwards, at line 144; the key-type checks at lines 148–160; the signature at
 881 line 165.

882 Two consequences follow. A token that will be rejected pays the discarded
 883 attempt in full: 18.04 μ s on the host, and 373.79 μ s on the runtime the studied
 884 service deploys (Table 4). And restricting permitted algorithms, the defence
 885 RFC 8725 recommends and which 94% of the call sites in our sample omit,
 886 does not avoid it, because the restriction is enforced after resolution. The cost
 887 is therefore under the control of any client that can reach the endpoint, not
 888 only of the service’s legitimate load. That is a statement about reachability,
 889 established from the source; its magnitude is the one measured in Sections 4–6.

890 8.3. The attempt arrived with a key-confusion fix

891 We compared `verify.js` in the published packages of versions 8.5.1, 9.0.0
 892 and 9.0.3. In 8.5.1 the secret is passed to the signature routine as given, with
 893 no key-object resolution. The resolution block of Figure 2 first appears in
 894 9.0.0 and is unchanged, apart from whitespace, through 9.0.3. Version 9.0.0
 895 is the release that fixed CVE-2022-23541 and CVE-2022-23539 (Auth0, 2022),
 896 and the checks implementing those fixes — the comparison of the declared
 897 algorithm with the key’s type at lines 148–152 and the validation of asymmetric

key types at lines 154–160 — operate on the key object the resolution block produces. The attempt is thus not an oversight in otherwise untouched code. It is part of a security fix, and its price is set by whichever OpenSSL release a deployment happens to bundle (Section 5).

This is consistent with the one earlier report of the symptom: HS256 verification in the 9.x series measured 34 times slower than in 8.5.1, on a Node release bundling OpenSSL 3.0 (Section 8.6). We did not time version 8.5.1 ourselves, and state the connection as consistent with the report rather than as established by it.

8.4. *A fix, and a constraint on it*

The fix is to dispatch on the token’s declared algorithm before attempting the asymmetric parse, so that an HMAC token with an ordinary string secret is resolved directly. Applied, this takes the call from 29.92 μ s to 7.208 μ s, a factor of 4.15, and to within 1.208 μ s of supplying a pre-parsed key, with no change in calling code. That residual is the median of the per-round paired differences, not the difference of the two medians; the two are not identical and the former is reported throughout.

We tested it against the library’s own suite rather than only against assertions of our own. Cloned at tag v9.0.3, the patch applies cleanly and the suite is unchanged: 511 passing and 0 failing unpatched, 511 and 0 patched.

These figures are from the host. The practical benefit is largest where the discarded parse is largest — of order 373.79 μ s on the runtime the service actually deploys (Table 4) — and we did not measure the patched library on the container runtimes. The reported factor of 4.15 is therefore a lower bound on the improvement available in the deployed environment, and we state it as a host result rather than as a general one.

The constraint, which is the more important half. The discarded attempt is not purely wasteful. The *success* of that parse on asymmetric key material is what produces a key object whose type a later check depends on, and that check is what prevents symmetric and asymmetric material from being interchanged (Sheffer et al., 2020). A change that dispatches on the declared algorithm while relaxing which key material may take the fast path alters which inputs reach that check.

The fix must therefore be narrow: the fast path is restricted to string secrets that cannot be asymmetric key material, so anything that might be resolves exactly as it does today. Removing that material restriction —

934 the form of the optimisation we argue against — produces 2 failures in the
935 library’s own suite, both in its key-confusion tests (“should not allow HMAC
936 verification with an RSA key in PEM format” and, under “signing with pub
937 key as symmetric”, “should not verify”). All of these outcomes — the suite
938 unchanged under the narrow patch, two key-confusion failures without its
939 guard — reproduce on the x86_64 machine of Section 3.7 under a different
940 Node and OpenSSL release (Katha et al., 2026).

941 *The existing suite already guards the property.* No new regression test is
942 needed: the library’s existing suite constrains the change, and a contributor
943 proposing the unrestricted optimisation would be caught by tests they are
944 required to run. The finding is therefore not a missing test but a hidden
945 coupling: the cost documented in Section 4 cannot be removed by the obvious
946 route, and the reason is a property the maintainers already test for but which
947 is not evident from the code being changed.

948 The independent defence against this class of confusion is for the caller to
949 restrict permitted algorithms (Sheffer et al., 2020), and 94% of the call sites
950 surveyed in Section 6.5 do not.

951 8.5. *What the narrow fix leaves reachable*

952 The narrow fix selects its fast path on the token’s declared algorithm,
953 and the declared algorithm is chosen by whoever constructs the token. For a
954 token declaring an HMAC algorithm and a plain string secret the parse is
955 skipped. For a token declaring any other algorithm the patch falls through
956 to the original resolution, so `createPublicKey()` is attempted on the shared
957 secret and fails exactly as it does today; the token is then rejected by the
958 algorithm list or the key-type check, but only after the parse. Table 10 sets
959 out which inputs reach the attempt under each configuration. The narrow
960 fix removes the cost from legitimate traffic, not from traffic an adversary
961 constructs. Only a pre-parsed key object removes it for every input, because
962 the resolution block is skipped entirely when the configured key is already a
963 `KeyObject` (line 120).

964 This follows from the code and was not separately timed: we did not
965 measure the patched library on tokens declaring a non-HMAC algorithm,
966 and the per-call cost such a token incurs is that of the discarded attempt
967 measured in Sections 4 and 5. A variant that dispatched on the *configured*
968 algorithms rather than the declared one would close the gap for callers who
969 restrict permitted algorithms to HMAC, since tokens declaring anything else

Table 10: Which tokens reach the discarded key-parse attempt, for a service configured with an HMAC shared secret, as read from the source at tag v9.0.3 and from the patch of Section 8.4. “Well-formed”: three segments, JSON header and payload, non-empty signature. No row requires knowledge of the secret.

Configuration	Tokens that reach the parse
Stock, string secret	every well-formed token, forged or genuine
Narrow fix, plain string secret (no PEM or JWK form)	every well-formed token whose header declares a non-HMAC algorithm; no such token can be valid for this service
Pre-parsed <code>KeyObject</code>	none

are rejected regardless; it is the choice `fast-jwt` makes (Section 7). We did not implement or test it, it would help only the minority of callers who restrict algorithms, and it would have to keep the same guard on key material.

8.6. Reported upstream

Before reporting, we searched the library’s issue tracker for earlier reports. One exists: an issue opened in April 2024¹ reports that HS256 verification in the 9.x series is 34 times slower than in 8.5.1, on a Node release that bundles OpenSSL 3.0. It diagnoses no cause and, at the time of writing, has no maintainer response. That is the symptom this paper explains, observed independently two years earlier by a user who could not see its source. We reported the mechanism, with a minimal reproduction, per-runtime measurements and the OpenSSL finding, as a separate issue², and a pull request implementing the narrow form of the fix, with its guard, is open against it. At the time of writing neither has received a maintainer response. We state that rather than omit it: the library is maintained at a slow cadence and a response may postdate publication.

9. Implications for practice

Pre-parse key material once, at startup. On the host this removes 23.92 μ s per validation [23.58, 24.00]; on the runtime the service actually deploys, where the discarded parse costs 373.79 μ s, it removes far more. It requires

¹<https://github.com/auth0/node-jsonwebtoken/issues/966>

²<https://github.com/auth0/node-jsonwebtoken/issues/1046>

no architectural change, no new component and no movement of a trust boundary, and it is the only remedy in this paper that removes the attempt for every input, including tokens an adversary constructs (Table 10). On a current runtime bundling OpenSSL 3.5 the saving is smaller — 24.56 μ s per validation in Table 9 — but it is not negligible beside the 347.3 μ s of processor time the deployed service spends on a request with validation disabled (Table 8). The two figures come from different machines and are compared as orders of magnitude only.

Treat the base image as part of the measurement. The same code, the same library and the same architecture differ by $25.36\times$ on this path across the container images measured, because they pin different OpenSSL versions. A performance figure quoted without the base image it was obtained on is not reproducible, and a regression appearing after a base-image bump may have nothing to do with the application. Section 5.5 shows that on this path the OpenSSL release is the cause; but the practical point would hold even if it were not, since it is enough that the image changes the number.

For library authors: do not use a failing parse as a type test. Section 7 found three designs that avoid the cost — inspect the material and dispatch, refuse ambiguous input at the interface, or type the key — and all three ship in maintained libraries today. What they share is that resolution happens once, or not at all, rather than on every call. The design that pays most is the one that asks an expensive parser to answer a cheap question on every request.

Pin the algorithm. Independently of anything here, callers should restrict permitted algorithms (Sheffer et al., 2020). Most of the call sites we surveyed do not, which leaves them relying entirely on library-internal key typing for a property they could assert directly in one argument. Pinning does not remove the cost measured here, which is incurred before the restriction is checked (Section 8.2); it is an independent defence, not a substitute for pre-parsing.

For security reviewers: review performance changes to verification as security changes. The cheapest-looking optimisation of this path fails the library’s key-confusion tests, and the property it breaks is not evident from the lines being changed (Section 8.4). Conversely, the security fix that introduced the path carries a cost that varies by more than an order of magnitude with the bundled OpenSSL release (Sections 5 and 8.3). Changes to token verification should be run against the library’s key-confusion tests and timed on the oldest cryptographic library a deployment may bundle.

9.1. Relocation is not the first remedy

A cost believed to be cryptographic invites relocation to a sidecar or gateway. The cost measured here is not cryptographic and is largely avoidable in place, so an architectural remedy that also moves a trust boundary should not be the first thing reached for. We implemented that relocation for the service studied here; applying a mesh validation policy duplicates verification rather than moving it, and making it effective requires the application to trust a proxy-supplied claims header — a known anti-pattern that needs a filter stripping client-supplied copies. We report that only as a design note: it carries no measurement, and the configuration, the observation and the inconclusive automated probe are documented in the replication package (Katha et al., 2026).

10. Threats to validity and future work

The limitations below are grouped by what they bear on: the measurements, the deployment evidence, the survey of public code, and the scope of the claims. Each states what the paper does not claim as a result.

10.1. Measurement

Two architectures, one of them shared. The primary measurements come from a single ARM64 host. Section 5.6 repeats them on x86_64, and there the split between OpenSSL series and the component ordering of the host decomposition hold. But the x86_64 machine is a virtual machine on shared cloud infrastructure, whose absolute timings are known to be unreliable (Laaber et al., 2019); we therefore take only orderings and ratios between interleaved conditions from it, and no magnitude in this paper should be read as holding on x86_64 hardware in general. A repetition on a dedicated x86_64 host, which the replication package scripts in one command, would turn the ordering check into a replication of magnitudes. The C probe, the exception baseline and the cross-library comparison were measured only on the x86_64 machine. Their conclusions rest on ratios between conditions measured in the same rounds, which is the kind of quantity a shared machine can support, and the effects they report are far larger than the machine’s noise: the median between-process coefficient of variation is 1.5%, 2.9% and 2.8% in the three runs. The largest values (28.0%, 27.7% and 20.2%) occur on sub-microsecond or JIT-dominated conditions that carry no claim, and medians of per-process medians are insensitive to them. The C probe needs no JavaScript runtime

1061 and the scripts rebuild every OpenSSL release from source, so repeating it on
1062 the ARM64 host or a dedicated x86_64 machine is one command.

1063 *The C probe reproduces Node’s calls, not Node’s build.* Section 5.5 issues the
1064 OpenSSL calls Node issues, as read from the Node source, against OpenSSL
1065 releases we built. Node builds its bundled OpenSSL with its own configuration,
1066 so the C figures approximate rather than reproduce the parse cost inside a
1067 given runtime, and the binding column of Table 3 inherits that approximation —
1068 it is slightly negative on two OpenSSL 3.0 runtimes for that reason. The causal
1069 claim does not inherit it: it compares releases under one build configuration
1070 with nothing else varying.

1071 *The exception baseline is a reconstruction.* Node constructs the error from
1072 C++ through V8’s API; our baseline constructs an error of the same shape,
1073 message and properties from JavaScript at a comparable stack depth. The
1074 two paths share V8’s error construction and stack capture, and the paired
1075 stack-capture measurement on the real probe agrees in order of magnitude
1076 with the synthetic one, but they are not the same code path. The ARM64
1077 host figure of Table 2 was not itself decomposed; Table 3 decomposes the
1078 same component on six other runtimes.

1079 *The headline host figure has no per-row OpenSSL version.* The host decom-
1080 position of Section 4 predates the per-measurement OpenSSL field. The
1081 version in force is known from the run timestamp and a note recorded at the
1082 time: OpenSSL 3.6.3, which a package upgrade later that day moved to 3.6.4
1083 under the same Node binary. We report it as inferred, marked as such in
1084 Table 4, rather than as recorded. Three measurements that do carry their
1085 version bound how much this matters. The same probe costs 18.01 μ s under
1086 Node 26.10.0 with OpenSSL 3.5.8, 19.42 μ s in the x86_64 host repetition with
1087 OpenSSL 3.5.5, and 16.58 μ s in the Node 26 container with OpenSSL 3.5.7 —
1088 all of the same order as 18.04 μ s, and all from OpenSSL releases after the
1089 step of Table 5. The headline figure is therefore representative of a post-3.2
1090 OpenSSL, which is all the paper uses it for; it would be wrong by more than
1091 an order of magnitude only if the host had run an OpenSSL 3.0 or 3.1 release,
1092 which the record excludes.

1093 10.2. Deployment measurement

1094 *Ranges of two, with a systematic component.* The in-situ dispersion is the
1095 range across an ascending and a descending pass. It is not a confidence

1096 interval, and Section 6.2 shows the two passes differ systematically rather
1097 than randomly, so they are not independent samples of a common value and
1098 the range understates uncertainty further. No inference in this paper depends
1099 on the width of those ranges.

1100 *One in-situ process, two passes.* The rate sweep rests on one process sweeping
1101 seven rates twice. That is sufficient for the claim made — the cost is present
1102 in deployment and of this order — and insufficient for any claim about the
1103 shape of the cost-versus-rate relation, which Section 6.1 accordingly does
1104 not make. The magnitude claim does not rest on the sweep alone: the
1105 A/B of Section 6.4 is a separate pair of windows, measured with a separate
1106 instrument, and it agrees with the microbenchmark prediction on processor
1107 time. The wall-clock excess over that prediction is attributed to time off
1108 the processor and not decomposed further; the decline with arrival rate has
1109 several candidate explanations unrelated to validation that we did not test
1110 (Section 6.1).

1111 *Host load, and the confound that motivated recording it.* A throughput curve
1112 measured on a host shared with its own load generator moves with unrelated
1113 activity on that host, as Section 3.10 describes, which is why no capacity claim
1114 appears in this paper. Every in-situ window here carries the load average
1115 observed during it, and Section 6.3 shows that window-to-window agreement
1116 tracks that load rather than the condition. Contention inflates a duration
1117 rather than deflating it, so in-situ figures are upper bounds; but comparisons
1118 between windows at different loads are not comparisons between conditions,
1119 and we do not treat them as such.

1120 *The A/B is two windows at unequal load.* The two arms of Table 8 are
1121 consecutive windows rather than interleaved rounds, and their host load
1122 is known only at the instant each began. That bars comparing them on
1123 wall-clock time, and we do not. It does not bar the CPU comparison the
1124 section rests on: contention inflates elapsed time, whereas a container’s own
1125 cgroup accounting charges it only the processor time it used. Attaching the
1126 per-second host sampler to a re-run of both arms, interleaved, would remove
1127 the limitation; we did not.

1128 10.3. Survey of public code

1129 *Sampling frames are not interchangeable.* The three prevalence instruments of
1130 Section 3.9 have different units and different selection mechanisms, and none

1131 supports a population proportion. The co-occurrence counts measure two
1132 strings appearing in one file, with a false-positive rate established at 5 of 13 on
1133 hand inspection. The classified sample has unknown inclusion probabilities, a
1134 limitation of repository search noted more generally by Kalliamvakou et al.
1135 (2014). The counter-search is selected on the outcome.

1136 *The matcher’s coverage.* Call sites are found by resolving the local binding
1137 of the import, which covers aliases, namespace imports, destructured and
1138 renamed imports and direct calls on the require expression. It does not resolve
1139 bindings across files, so a project that constructs a pre-parsed key in one file
1140 and validates in another is invisible to this analysis. This is the strongest
1141 threat to Section 6.5; within-file tracing found no such case, which is weak
1142 evidence against it.

1143 *One rater.* The 13 automatically flagged call sites were read and classified
1144 by the first author, and the authors are not disinterested: the paper’s claim
1145 is stronger the more of them are false positives, and 5 were classified that
1146 way. There was no second rater and no inter-rater agreement to report. The
1147 per-case verdicts, the evidence for each and the hashes of the files they were
1148 read from are published so the adjudication can be repeated, which is weaker
1149 than having repeated it.

1150 *10.4. Scope*

1151 *Token caching is not in the comparison.* A service that caches verified tokens
1152 pays this cost once per token rather than once per request. We did not
1153 measure such a configuration, so the per-request figures here do not describe
1154 a caching deployment.

1155 *The security analysis is from source.* Section 8 establishes reachability and
1156 provenance by reading `verify.js` at the releases named and by running the
1157 library’s own tests; it does not drive forged tokens at the deployed service,
1158 time the patched library on tokens declaring a non-HMAC algorithm, or
1159 time version 8.5.1. Its statements are therefore about which inputs reach the
1160 attempt and when the code arrived, not about throughput under attack, and
1161 the paper makes no availability claim.

1162 *Seven libraries, one version each.* Section 7 covers seven libraries in four
1163 languages, chosen as the most used in each ecosystem, each at one release.
1164 Other libraries, and other releases of these, may resolve keys differently; each

1165 classification is from source at the version measured. The comparison supports
1166 the claim that the cost follows a design choice rather than JWT validation; it
1167 does not estimate how many libraries make that choice.

1168 10.5. Future work

1169 Four directions follow from the limitations above.

1170 *Security.* Section 8 establishes which inputs reach the discarded attempt; it
1171 does not measure what an adversary can do with it. Driving forged tokens
1172 at a deployed service, under the stock library and under the narrow fix,
1173 would turn that reachability statement into an availability measurement. The
1174 configured-algorithm variant of the fix (Section 8.5) should be implemented
1175 and run against the library’s key-confusion tests, and a fix that removes the
1176 attempt for every input without requiring callers to pre-parse keys remains
1177 to be designed and proposed upstream.

1178 *Cause.* The C probe locates the fall in two steps, at 3.1 and 3.2 (Section 5.5).
1179 Bisecting the OpenSSL history between those releases would identify the
1180 changes responsible, and timing version 8.5.1 of the library would test the
1181 connection to the earlier upstream report directly (Section 8.3).

1182 *Generality.* Seven libraries at one version each show that the cost follows a
1183 design choice; they do not estimate how often the choice is made. A systematic
1184 survey of JWT and JOSE libraries, and of other authentication paths that
1185 resolve key material on every call, would. Resolving call sites across files and
1186 adding a second rater would strengthen the prevalence evidence of Section 6.5.

1187 *Deployment.* A dedicated x86_64 repetition, a Linux-native baseline for con-
1188 tainerisation, interleaved A/B windows with per-second host sampling, a
1189 controlled study of the decline with arrival rate, and a deployment that caches
1190 verified tokens would each close one limitation stated above. The scripts for
1191 the first are already in the replication package.

1192 11. Conclusion

1193 The per-request cost of token validation in this library is real, is worth
1194 attending to, and is not the signature: the primitive is 2.208 μ s of a 29.92 μ s
1195 call. It is not key conversion either, which costs 0.583 μ s. It is a parse
1196 attempted in the wrong order, whose failure is constructed and thrown away

1197 on every request at $18.04\ \mu\text{s}$, 60% of the call. Where that attempt is expensive
1198 it is almost entirely OpenSSL work; the exception that ends it costs a few
1199 microseconds.

1200 It is also a security property of the path. The attempt runs before the
1201 algorithm check and the signature, so any well-formed token can cause it; it
1202 entered the library with a key-confusion fix and that fix depends on it, so
1203 removing it naively fails the library’s key-confusion tests; and the narrow fix
1204 that keeps the defence still leaves it reachable through an attacker-chosen
1205 algorithm header. Pre-parsing the key once, at startup, is both the fastest
1206 configuration and the only one no input can push onto the slow path.

1207 Its price is set by the OpenSSL release a base image happens to pin,
1208 varying $25.36\times$ across runtimes while the rest of the validation path varies
1209 by $1.26\times$ — and set causally, not only by association: the same calls from C
1210 cost $394.4\ \mu\text{s}$ on OpenSSL 3.0.16 and $8.447\ \mu\text{s}$ on 3.5.8, and a newer engine
1211 paired with an older OpenSSL is the slow runtime, not the fast one.

1212 It is a design choice, not a property of JWT: of six other libraries in four
1213 languages none attempts the parse, five pay nothing for a raw secret, and the
1214 one that re-imports its key on every call pays an order of magnitude less. The
1215 symptom had been reported upstream two years before this study without a
1216 diagnosis; we have reported the mechanism.

1217 The pattern that avoids the cost is rare in our sample: among 112
1218 repositories that already reference the pre-parsing interface and call the
1219 validator, 6 use it. No population proportion follows.

1220 Two explanations that present themselves before the measurement are
1221 refuted by it. That the dominant cost is key conversion is refuted by measur-
1222 ing the conversion. That a throughput collapse under layered enforcement
1223 characterises the service is refuted by the collapse point moving with unrelated
1224 host activity, which is why no capacity claim appears here. Both have the
1225 same shape, and it is the shape this paper’s methodology is built to avoid:
1226 a cost attributed to the variable under study while a second variable moves
1227 alongside it, unrecorded at the granularity at which it could change. The
1228 bundled OpenSSL version in Section 5 is the same hazard caught in time —
1229 the quantity varies $25.36\times$ with a component most measurement protocols
1230 record once per run, if at all. The lesson is not that one should measure
1231 in deployment. It is narrower: record the identity of the component whose
1232 behaviour is being measured, on every measurement, at the granularity at
1233 which it can change.

1234 **CRedit authorship contribution statement**

1235 **Jannatul Ferdous Katha:** Conceptualization, Methodology, Software,
1236 Investigation, Data curation, Formal analysis, Visualization, Writing – original
1237 draft. **Tasmia Tahmid Prova:** Validation, Writing – review & editing. **Md**
1238 **Kishor Morol:** Supervision, Writing – review & editing. **Tze Hui Liew:**
1239 Supervision, Project administration, Writing – review & editing. **Dip Nandi:**
1240 Supervision, Writing – review & editing.

1241 **Declaration of competing interest**

1242 The authors declare that they have no known competing financial interests
1243 or personal relationships that could have appeared to influence the work
1244 reported in this paper.

1245 **Funding**

1246 This research did not receive any specific grant from funding agencies in
1247 the public, commercial, or not-for-profit sectors.

1248 **Data availability**

1249 The data supporting the results of this study are openly available
1250 in the replication package (Katha et al., 2026) at [https://github.com/](https://github.com/Katha-Sikdar/microservices-api)
1251 [Katha-Sikdar/microservices-api](https://github.com/Katha-Sikdar/microservices-api), directory `psao-artifact/`. It contains
1252 all measurements, the instrumentation, the analysis code and the scripts that
1253 generate every numeric value and every figure in this paper, including the
1254 C program and the script that builds each OpenSSL release, the exception
1255 baseline, and the cross-library harnesses in four languages with their pinned
1256 dependency manifests. Every number is generated from a file under the
1257 run or survey directories; none is entered by hand. The survey corpus is
1258 third-party source and is not redistributed; a manifest pinning every file
1259 examined by content hash stands in for it, covering 502 files across 457
1260 repositories. Content hashes resolved for all of them; the repository HEAD
1261 commit resolved for 501 of 502, one repository having become unavailable
1262 between fetch and manifest. The content hash is the authoritative pin and is
1263 complete.

Ethics and responsible disclosure

This study measures software behaviour and publicly available source code; it involves no human participants and no personal data. No attack was run against any system the authors do not control. The key-confusion behaviour discussed in Section 8 is an existing, publicly documented attack class tested only through the library’s own test suite; the upstream report describes the constraint at the level of mechanism and does not include an exploitation recipe.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) in order to restructure the manuscript for this journal’s format and to draft revisions of its text. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

- Acar, Y., Backes, M., Fahl, S., Garfinkel, S., Kim, D., Mazurek, M.L., Stransky, C., 2017. Comparing the usability of cryptographic APIs, in: 2017 IEEE Symposium on Security and Privacy (SP). doi:10.1109/SP.2017.52.
- Auth0, 2022. CVE-2022-23539, CVE-2022-23541, CVE-2022-23540: Security update for jsonwebtoken. Auth0 security bulletin, 21 December 2022, <https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jwebtoken>. Accessed 2026-09-28.
- Barrett, E., Bolz-Tereick, C.F., Killick, R., Mount, S., Tratt, L., 2017. Virtual machine warmup blows hot and cold. Proceedings of the ACM on Programming Languages 1, 52:1–52:27. doi:10.1145/3133876.
- Cabral, B., Marques, P., 2007. Exception handling: A field study in Java and .NET, in: ECOOP 2007 – Object-Oriented Programming, Springer. pp. 151–175. doi:10.1007/978-3-540-73589-2_8.

1293 Chen, J., Shang, W., 2017. An exploratory study of performance regression
1294 introducing code changes, in: 2017 IEEE International Conference on
1295 Software Maintenance and Evolution (ICSME). doi:10.1109/ICSME.2017.
1296 13.

1297 Cito, J., Schermann, G., Wittern, J.E., Leitner, P., Zumberi, S., Gall, H.C.,
1298 2017. An empirical analysis of the Docker container ecosystem on GitHub,
1299 in: 2017 IEEE/ACM 14th International Conference on Mining Software
1300 Repositories (MSR). doi:10.1109/MSR.2017.67.

1301 Coarfa, C., Druschel, P., Wallach, D.S., 2006. Performance analysis of TLS
1302 web servers. ACM Transactions on Computer Systems 24. doi:10.1145/
1303 1124153.1124155.

1304 Costa, D., Bezemer, C.P., Leitner, P., Andrzejak, A., 2021. What’s wrong
1305 with my benchmark results? Studying bad practices in JMH benchmarks.
1306 IEEE Transactions on Software Engineering 47, 1452–1467. doi:10.1109/
1307 TSE.2019.2925345.

1308 Curtsinger, C., Berger, E.D., 2013. STABILIZER: Statistically sound perfor-
1309 mance evaluation, in: Proceedings of the 18th International Conference on
1310 Architectural Support for Programming Languages and Operating Systems
1311 (ASPLOS ’13), pp. 219–228. doi:10.1145/2451116.2451141.

1312 Davis, J.C., Williamson, E.R., Lee, D., 2018. A sense of time for JavaScript
1313 and Node.js: First-class timeouts as a cure for event handler poisoning, in:
1314 Proceedings of the 27th USENIX Security Symposium, pp. 343–359.

1315 Detering, D., Somorovsky, J., Mainka, C., Mladenov, V., Schwenk, J., 2017.
1316 On the (in-)security of JavaScript object signing and encryption, in: Pro-
1317 ceedings of the 1st Reversing and Offensive-oriented Trends Symposium
1318 (ROOTS ’17). doi:10.1145/3150376.3150379.

1319 Egele, M., Brumley, D., Fratantonio, Y., Kruegel, C., 2013. An empirical
1320 study of cryptographic misuse in Android applications, in: Proceedings of
1321 the 2013 ACM SIGSAC Conference on Computer and Communications
1322 Security (CCS ’13). doi:10.1145/2508859.2516693.

1323 Gaynor, A., Kehrer, P., 2026. The state of OpenSSL for pyca/cryptogra-
1324 phy. pyca/cryptography project statement, <https://cryptography.io/>

1325 `en/latest/statements/state-of-openssl/`. Maintainer statement, not
1326 peer reviewed; accessed 2026-09-24.

1327 Georges, A., Buytaert, D., Eeckhout, L., 2007. Statistically rigorous Java
1328 performance evaluation, in: Proceedings of the 22nd Annual ACM SIG-
1329 PLAN Conference on Object-Oriented Programming Systems, Languages
1330 and Applications (OOPSLA '07). doi:10.1145/1297027.1297033.

1331 Georgiev, M., Iyengar, S., Jana, S., Anubhai, R., Boneh, D., Shmatikov, V.,
1332 2012. The most dangerous code in the world: Validating SSL certificates
1333 in non-browser software, in: Proceedings of the 2012 ACM Conference on
1334 Computer and Communications Security (CCS '12). doi:10.1145/2382196.
1335 2382204.

1336 GitHub Advisory Database, 2015. Verification bypass in jsonwebtoken (CVE-
1337 2015-9235, GHSA-c7hr-j4mj-j2w6). [https://github.com/advisories/](https://github.com/advisories/GHSA-c7hr-j4mj-j2w6)
1338 [GHSA-c7hr-j4mj-j2w6](https://github.com/advisories/GHSA-c7hr-j4mj-j2w6). Accessed 2026-09-28.

1339 Gong, L., Pradel, M., Sen, K., 2015. JITProf: Pinpointing JIT-unfriendly
1340 JavaScript code, in: Proceedings of the 10th Joint Meeting on Foundations
1341 of Software Engineering (ESEC/FSE '15). doi:10.1145/2786805.2786831.

1342 Green, M., Smith, M., 2016. Developers are not the enemy! the need for usable
1343 security APIs. IEEE Security & Privacy 14. doi:10.1109/MSP.2016.111.

1344 Indela, S., Kulkarni, M., Nayak, K., Dumitras, T., 2016. Helping Johnny
1345 encrypt: Toward semantic interfaces for cryptographic frameworks, in:
1346 Proceedings of the 2016 ACM International Symposium on New Ideas, New
1347 Paradigms, and Reflections on Programming and Software (Onward! 2016).
1348 doi:10.1145/2986012.2986024.

1349 Jin, G., Song, L., Shi, X., Scherpelz, J., Lu, S., 2012. Understanding and
1350 detecting real-world performance bugs, in: Proceedings of the 33rd ACM
1351 SIGPLAN Conference on Programming Language Design and Implementa-
1352 tion (PLDI '12), pp. 77–88. doi:10.1145/2254064.2254075.

1353 Jones, M.B., Bradley, J., Sakimura, N., 2015. JSON Web Token (JWT).
1354 Request for Comments RFC 7519. Internet Engineering Task Force. doi:10.
1355 17487/RFC7519.

1356 Kalibera, T., Jones, R., 2013. Rigorous benchmarking in reasonable time, in:
1357 Proceedings of the 2013 International Symposium on Memory Management
1358 (ISMM '13). doi:10.1145/2464157.2464160.

1359 Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian,
1360 D., 2014. The promises and perils of mining GitHub, in: Proceedings of
1361 the 11th Working Conference on Mining Software Repositories (MSR 2014).
1362 doi:10.1145/2597073.2597074.

1363 Kaltenecker, C., Mühlbauer, S., Grebhahn, A., Siegmund, N., Apel, S., 2023.
1364 Performance evolution of configurable software systems: An empirical study.
1365 Empirical Software Engineering 28. doi:10.1007/s10664-023-10338-3.

1366 Katha, J.F., Prova, T.T., Morol, M.K., Liew, T.H., Nandi, D., 2026.
1367 Replication package: where the cost of JWT verification actually
1368 goes. <https://github.com/Katha-Sikdar/microservices-api>. Direc-
1369 tory psao-artifact/.

1370 Kery, M.B., Le Goues, C., Myers, B.A., 2016. Examining programmer
1371 practices for locally handling exceptions, in: Proceedings of the 13th
1372 International Conference on Mining Software Repositories (MSR '16), pp.
1373 484–487. doi:10.1145/2901739.2903497.

1374 Krüger, S., Späth, J., Ali, K., Bodden, E., Mezini, M., 2018. CrySL: An
1375 extensible approach to validating the correct usage of cryptographic APIs,
1376 in: 32nd European Conference on Object-Oriented Programming (ECOOP
1377 2018), pp. 10:1–10:27. doi:10.4230/LIPIcs.ECOOP.2018.10.

1378 Laaber, C., Leitner, P., 2018. An evaluation of open-source software micro-
1379 benchmark suites for continuous performance assessment, in: Proceedings
1380 of the 15th International Conference on Mining Software Repositories (MSR
1381 '18). doi:10.1145/3196398.3196407.

1382 Laaber, C., Scheuner, J., Leitner, P., 2019. Software microbenchmarking
1383 in the cloud. how bad is it really? Empirical Software Engineering 24.
1384 doi:10.1007/s10664-019-09681-1.

1385 Lazar, D., Chen, H., Wang, X., Zeldovich, N., 2014. Why does cryptographic
1386 software fail? A case study and open problems, in: Proceedings of the
1387 5th Asia-Pacific Workshop on Systems (APSys '14). doi:10.1145/2637166.
1388 2637237.

1389 Leitner, P., Cito, J., 2016. Patterns in the chaos—a study of performance
1390 variation and predictability in public IaaS clouds. *ACM Transactions on*
1391 *Internet Technology* 16. doi:10.1145/2885497.

1392 Liu, Y., Xu, C., Cheung, S.C., 2014. Characterizing and detecting performance
1393 bugs for smartphone applications, in: *Proceedings of the 36th International*
1394 *Conference on Software Engineering (ICSE '14)*, pp. 1013–1024. doi:10.
1395 1145/2568225.2568229.

1396 Malka, J., Zacchiroli, S., Zimmermann, T., 2026. Docker does not guarantee
1397 reproducibility. *arXiv preprint arXiv:2601.12811*. No DOI registered;
1398 verified against the arXiv record.

1399 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2009. Producing
1400 wrong data without doing anything obviously wrong!, in: *Proceedings of the*
1401 *14th International Conference on Architectural Support for Programming*
1402 *Languages and Operating Systems (ASPLOS '09)*. doi:10.1145/1508244.
1403 1508275.

1404 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2010. Evaluating
1405 the accuracy of Java profilers, in: *Proceedings of the 31st ACM SIGPLAN*
1406 *Conference on Programming Language Design and Implementation (PLDI*
1407 *'10)*, pp. 187–197. doi:10.1145/1806596.1806618.

1408 Nadi, S., Krüger, S., Mezini, M., Bodden, E., 2016. Jumping through hoops, in:
1409 *Proceedings of the 38th International Conference on Software Engineering*
1410 *(ICSE '16)*. doi:10.1145/2884781.2884790.

1411 Naser, A., Sadeghi Lahijani, M., Wu, C., Gavahi, M., Hoang, V.T., Wang,
1412 Z., Yuan, X., 2020. Performance evaluation and modeling of cryptographic
1413 libraries for MPI communications. *arXiv preprint arXiv:2010.06139*. No
1414 DOI registered; verified against the arXiv record.

1415 Nistor, A., Jiang, T., Tan, L., 2013a. Discovering, reporting, and fixing
1416 performance bugs, in: *Proceedings of the 10th Working Conference on*
1417 *Mining Software Repositories (MSR '13)*. doi:10.1109/MSR.2013.6624035.

1418 Nistor, A., Song, L., Marinov, D., Lu, S., 2013b. Toddler: Detecting perfor-
1419 mance problems via similar memory-access patterns, in: *Proceedings of*
1420 *the 35th International Conference on Software Engineering (ICSE '13)*, pp.
1421 562–571. doi:10.1109/ICSE.2013.6606602.

1422 Ogasawara, T., Komatsu, H., Nakatani, T., 2006. EDO: Exception-directed
1423 optimization in Java. *ACM Transactions on Programming Languages and*
1424 *Systems* 28. doi:10.1145/1111596.1111598.

1425 de Pádua, G.B., Shang, W., 2017. Studying the prevalence of exception
1426 handling anti-patterns, in: *Proceedings of the 25th International Conference*
1427 *on Program Comprehension (ICPC '17)*, pp. 328–331. doi:10.1109/ICPC.
1428 2017.1.

1429 Parravicini, A., Mueller, T., 2021. The cost of speculation: Revisiting over-
1430 heads in the V8 JavaScript engine, in: *2021 IEEE International Symposium*
1431 *on Workload Characterization (IISWC)*. doi:10.1109/IISWC53511.2021.
1432 00013.

1433 Rahaman, S., Xiao, Y., Afrose, S., Shaon, F., Tian, K., Frantz, M., Kantar-
1434 cioglu, M., Yao, D.D., 2019. CryptoGuard: High precision detection of
1435 cryptographic vulnerabilities in massive-sized Java projects, in: *Proceedings*
1436 *of the 2019 ACM SIGSAC Conference on Computer and Communications*
1437 *Security (CCS '19)*, pp. 2455–2472. doi:10.1145/3319535.3345659.

1438 Renwick, J., Spink, T., Franke, B., 2019. Low-cost deterministic C++ ex-
1439 ceptions for embedded systems, in: *Proceedings of the 28th International*
1440 *Conference on Compiler Construction (CC 2019)*. doi:10.1145/3302516.
1441 3307346.

1442 Selakovic, M., Glaser, T., Pradel, M., 2017. An actionable performance
1443 profiler for optimizing the order of evaluations, in: *Proceedings of the*
1444 *26th ACM SIGSOFT International Symposium on Software Testing and*
1445 *Analysis (ISSTA '17)*. doi:10.1145/3092703.3092716.

1446 Selakovic, M., Pradel, M., 2016. Performance issues and optimizations in
1447 JavaScript: An empirical study, in: *Proceedings of the 38th International*
1448 *Conference on Software Engineering (ICSE '16)*, pp. 61–72. doi:10.1145/
1449 2884781.2884829.

1450 Shatnawi, A., Al-Duwairi, B., Smarneh, A., 2024. Comprehensive empirical
1451 study of Python JWT libraries. *Procedia Computer Science* 238, 827–832.
1452 doi:10.1016/j.procs.2024.06.099.

1453 Sheffer, Y., Hardt, D., Jones, M.B., 2020. JSON Web Token Best Current
1454 Practices. Request for Comments RFC 8725. Internet Engineering Task
1455 Force. doi:10.17487/RFC8725.

1456 Sikeridis, D., Kampanakis, P., Devetsikiotis, M., 2020. Assessing the overhead
1457 of post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the
1458 16th International Conference on Emerging Networking EXperiments and
1459 Technologies (CoNEXT '20). doi:10.1145/3386367.3431305.

1460 Song, L., Lu, S., 2014. Statistical debugging for real-world performance
1461 problems, in: Proceedings of the 2014 ACM International Conference
1462 on Object Oriented Programming Systems Languages & Applications
1463 (OOPSLA '14), pp. 561–578. doi:10.1145/2660193.2660234.

1464 Staicu, C.A., Pradel, M., 2018. Freezing the web: A study of ReDoS vul-
1465 nerabilities in JavaScript-based web servers, in: Proceedings of the 27th
1466 USENIX Security Symposium, pp. 361–376.

1467 Stefan, P., Horký, V., Bulej, L., Tůma, P., 2017. Unit testing performance in
1468 Java projects: Are we there yet?, in: Proceedings of the 8th ACM/SPEC
1469 International Conference on Performance Engineering (ICPE '17), pp. 401–
1470 412. doi:10.1145/3030207.3030226.

1471 Traini, L., Cortellessa, V., Di Pompeo, D., Tucci, M., 2023. Towards effective
1472 assessment of steady state performance in Java software: Are we there yet?
1473 Empirical Software Engineering 28. doi:10.1007/s10664-022-10247-x.

1474 Tuveri, N., Brumley, B.B., 2019. Start your ENGINes: Dynamically loadable
1475 contemporary crypto, in: 2019 IEEE Cybersecurity Development (SecDev).
1476 doi:10.1109/SecDev.2019.00014.

1477 Weimer, W., Necula, G.C., 2004. Finding and preventing run-time error
1478 handling mistakes, in: Proceedings of the 19th Annual ACM SIGPLAN
1479 Conference on Object-Oriented Programming, Systems, Languages, and
1480 Applications (OOPSLA '04). doi:10.1145/1028976.1029011.

1481 Xu, B., Jia, S., Lin, J., Zheng, F., Ma, Y., Liu, L., Gu, X., Song, L.,
1482 2023. JWTKey: Automatic cryptographic vulnerability detection in JWT
1483 applications, in: Computer Security – ESORICS 2023, Springer. doi:10.
1484 1007/978-3-031-51479-1_14.

1485 Yang, J., Wang, E., Chen, J., Wang, Q., Zhang, Y., Duan, H., Xie, W.,
1486 Wang, B., 2026. Token time bomb: Evaluating JWT implementations for
1487 vulnerability discovery, in: Proceedings of the Network and Distributed
1488 System Security Symposium (NDSS '26).

1489 Zaman, S., Adams, B., Hassan, A.E., 2012. A qualitative study on performance
1490 bugs, in: Proceedings of the 9th IEEE Working Conference on Mining
1491 Software Repositories (MSR '12), pp. 199–208. doi:10.1109/MSR.2012.
1492 6224281.

1493 Zerouali, A., Mens, T., Robles, G., Gonzalez-Barahona, J.M., 2019. On
1494 the relation between outdated Docker containers, severity vulnerabilities,
1495 and bugs, in: Proceedings of the 26th IEEE International Conference on
1496 Software Analysis, Evolution and Reengineering (SANER '19), pp. 491–501.
1497 doi:10.1109/SANER.2019.8668013.

1498 Zhu, X., She, G., Xue, B., Zhang, Y., Zhang, Y., Zou, X.K., Duan, X.,
1499 He, P., Krishnamurthy, A., Lentz, M., Zhuo, D., Mahajan, R., 2023.
1500 Dissecting overheads of service mesh sidecars, in: Proceedings of the 2023
1501 ACM Symposium on Cloud Computing (SoCC '23). doi:10.1145/3620678.
1502 3624652.